

RAJALAKSHMI ENGINEERING COLLEGE

CS23333
Object Oriented Programming Using JAVA

Contents

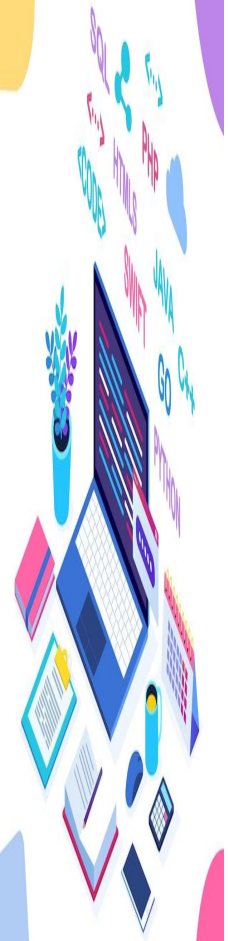
UNIT-I INTRODUCTION TO OOP AND JAVA FUNDAMENTALS

01 Introduction to OOP

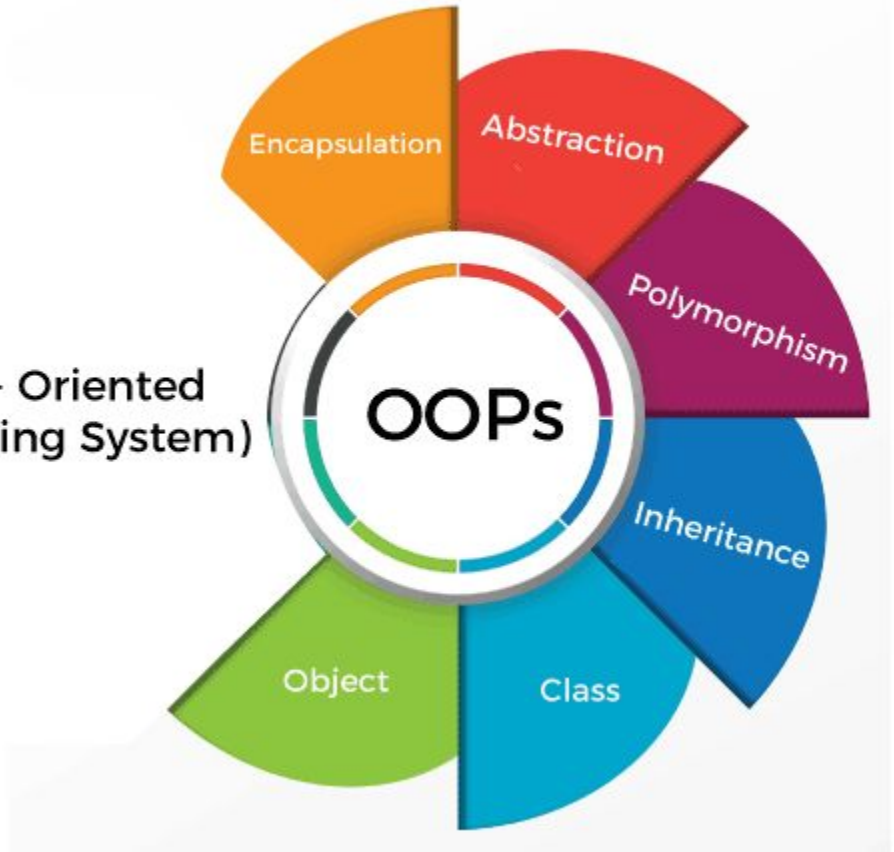
02 Overview of Java

03 Java Architecture

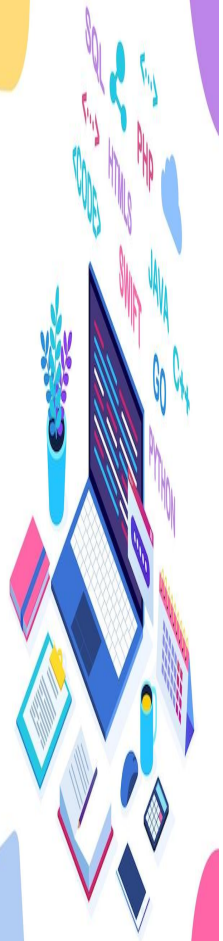
04 Data types, Variables and Operators



INTRODUCTION TO OOP

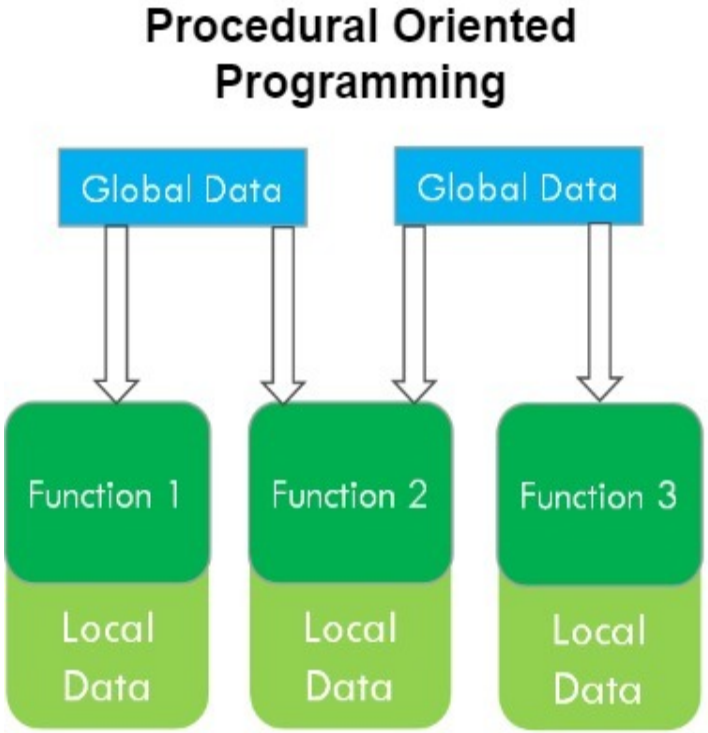


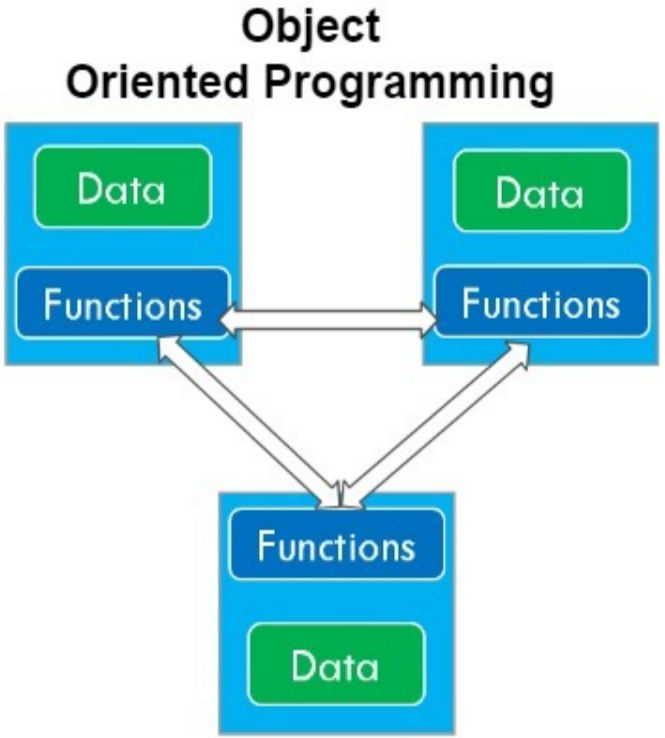
Java is an Object-Oriented Programming Language

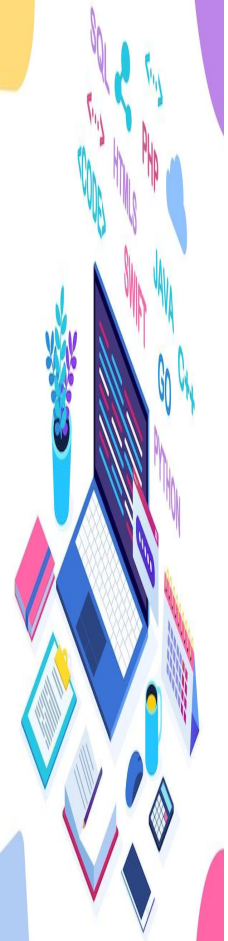


INTRODUCTION TO OOP

Transition from procedure-oriented programming –why?
Even it gives all solutions



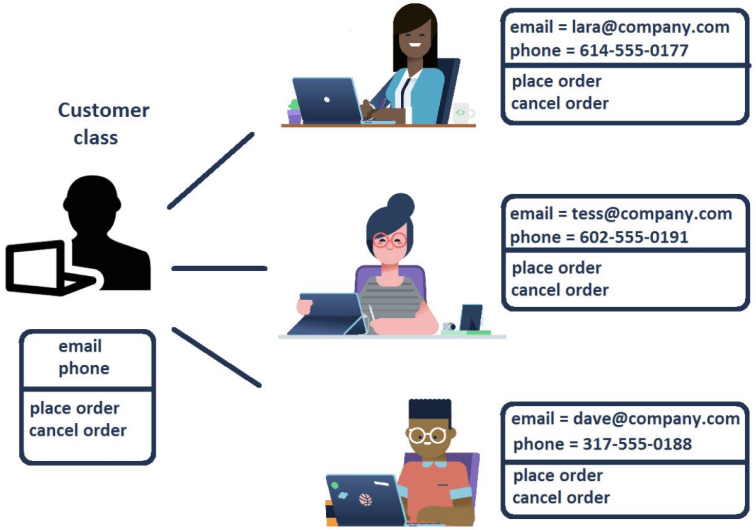




INTRODUCTION TO OOP

Class and Object

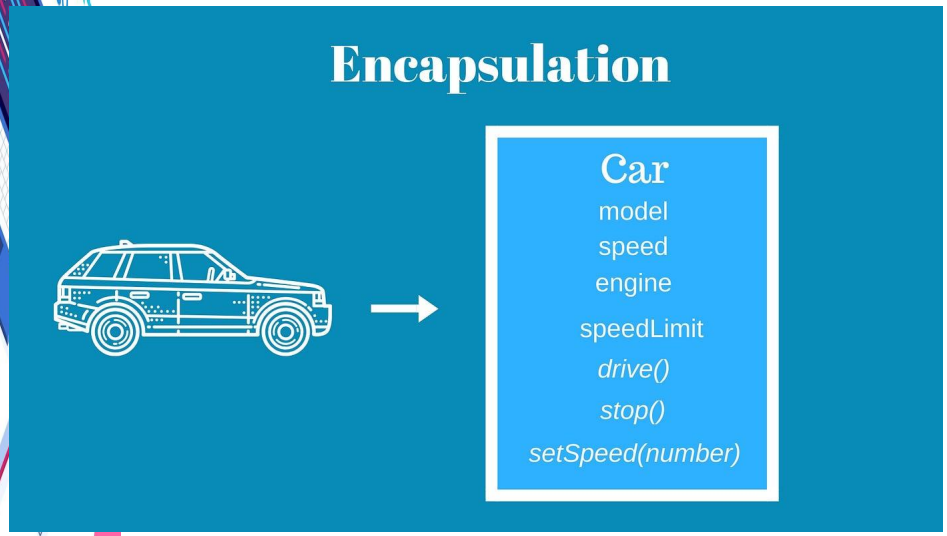
- A class is a template to create objects having similar properties and behavior, or in other words, we can say that a class is a blueprint for objects.
- Objects are the instances of a class that are created to use the attributes and methods of a class.

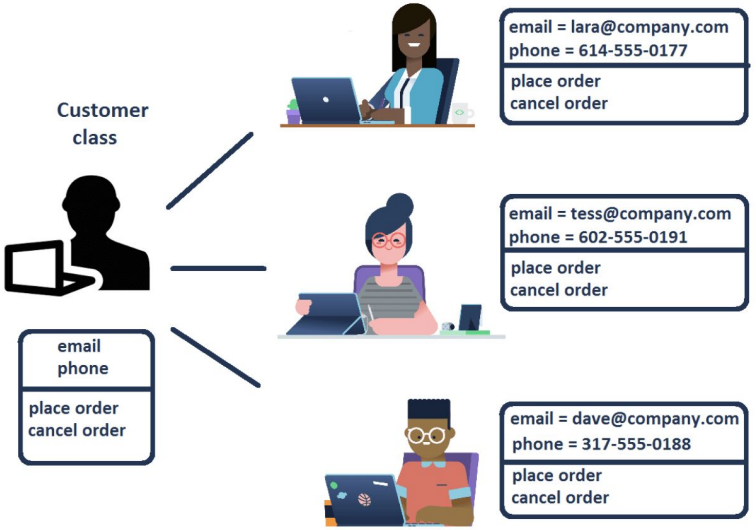


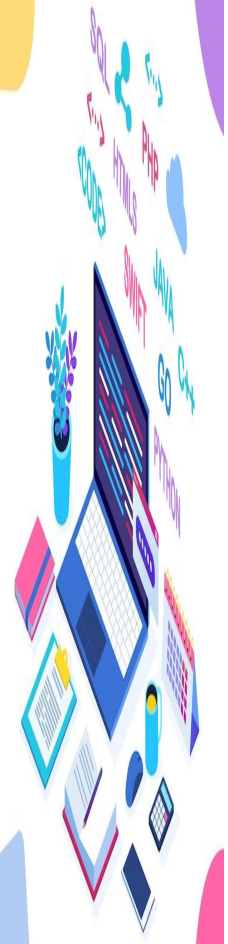
INTRODUCTION TO OOP

ENCAPSULATION

- Wrapping up of data and method in a single unit (Class)





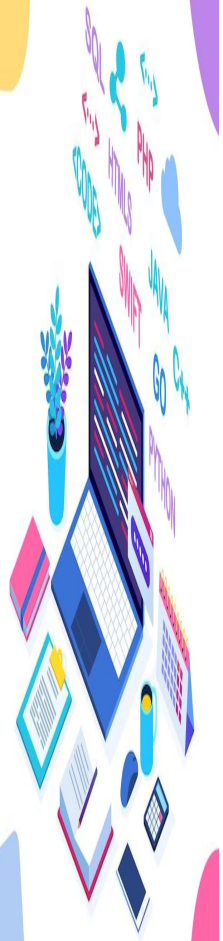


INTRODUCTION TO OOP

Abstraction

- Hiding the background details and saving the essential details
- Without that feature it can’t exist
- Hiding internal data and implementation from the outer world

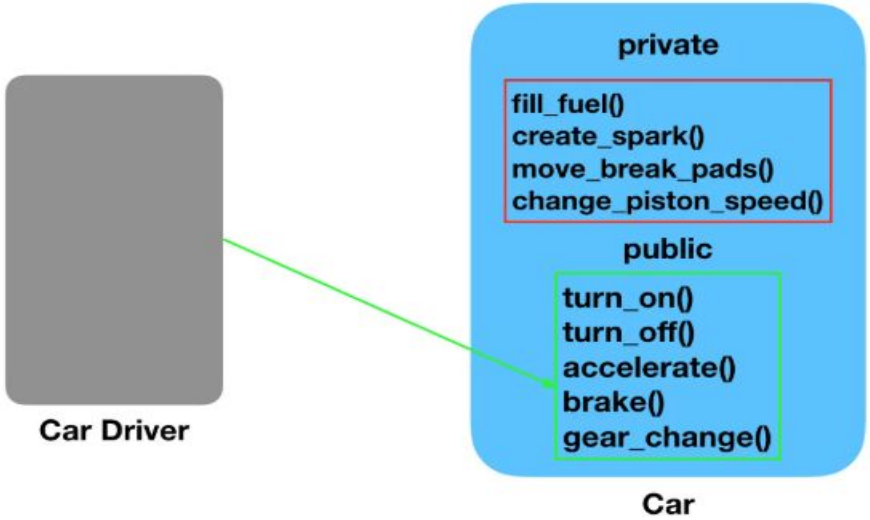
Your automobile is an excellent illustration of abstraction. Turning the key or pressing the start button is how you start a car. You don't need to know how your car's engine starts or what components it contains. The user is completely unaware of the car's internal implementation and complicated logic.

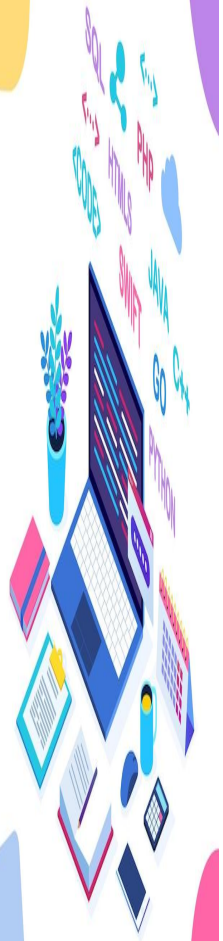


INTRODUCTION TO OOP

Abstraction

- Hiding the background details and saving the essential details
- Without that feature it can't exist
- Hiding internal data and implementation from the outer world

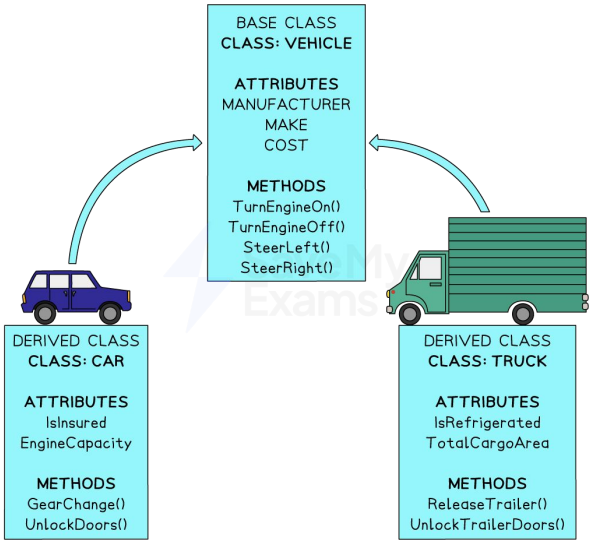


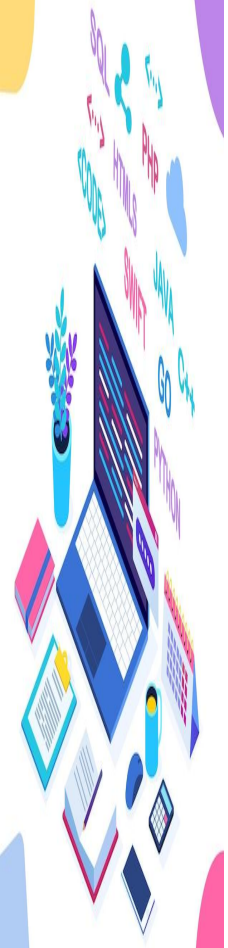


INTRODUCTION TO OOP

Inheritance

- In Java, inheritance means creating new classes based on existing ones. A class that inherits from another class can reuse the methods and fields of that class.





INTRODUCTION TO OOP

Inheritance

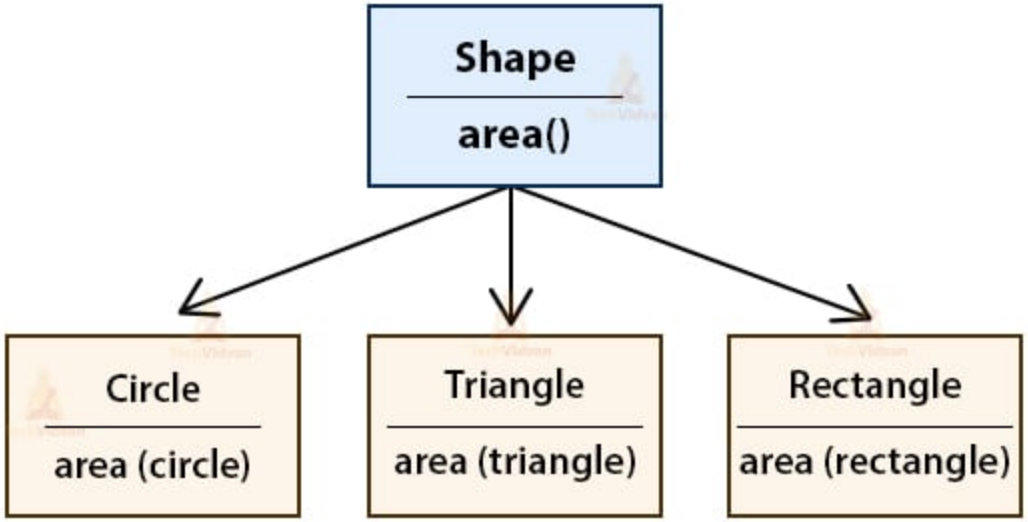
- **Super Class:** The class whose features are inherited is known as superclass(or a base class or a parent class).
- **Sub Class:** The class that inherits the other class is known as a subclass(or a derived class, extended class, or child class). The subclass can add its own fields and methods in addition to the superclass fields and methods.
- **Reusability:** Inheritance supports the concept of "reusability", i.e. when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class. By doing this, we



INTRODUCTION TO OOP

POLYMORPHISM

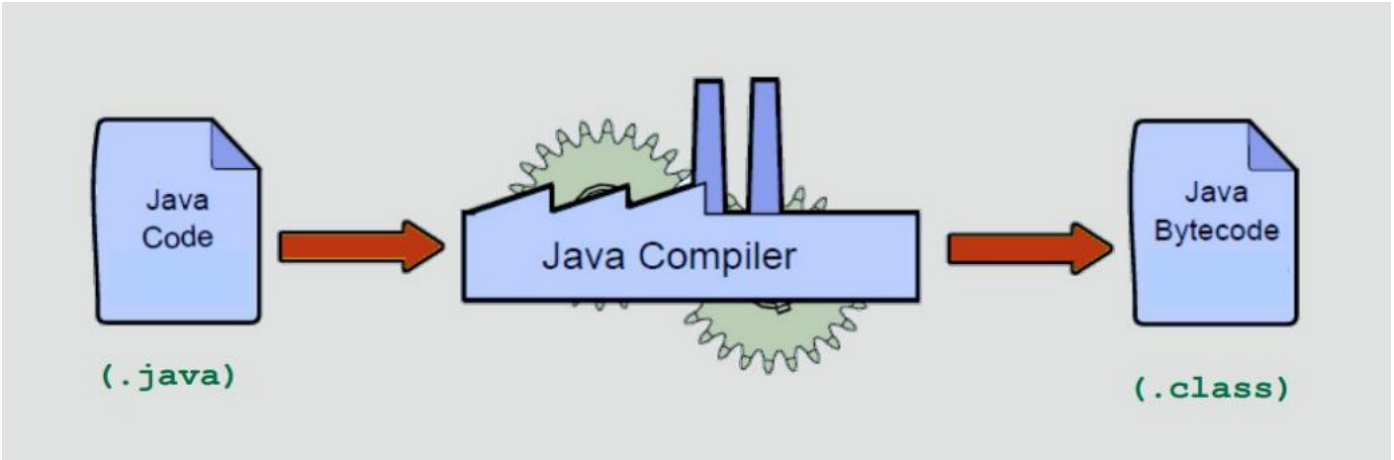
- Ability to take many forms

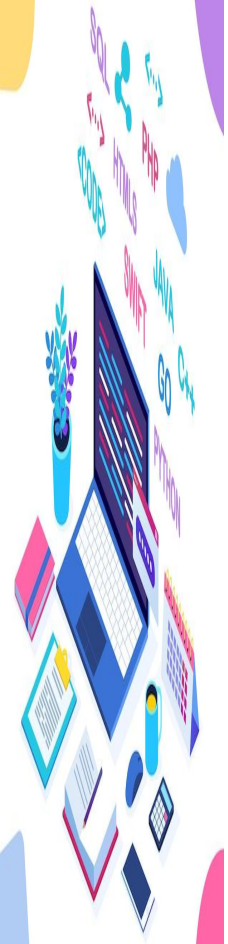




WHY JAVA?

- Java follows the principle of Write Once, Run Anywhere (WORA). This means Java programs are compiled into an intermediate form called bytecode, which can be executed on any machine that has a JVM, regardless of the operating system.

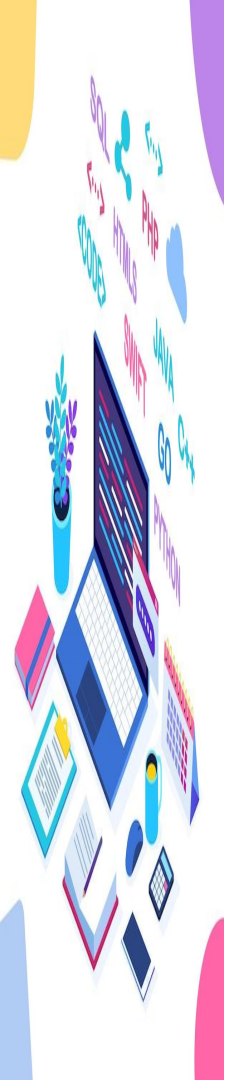




OVERVIEW OF JAVA

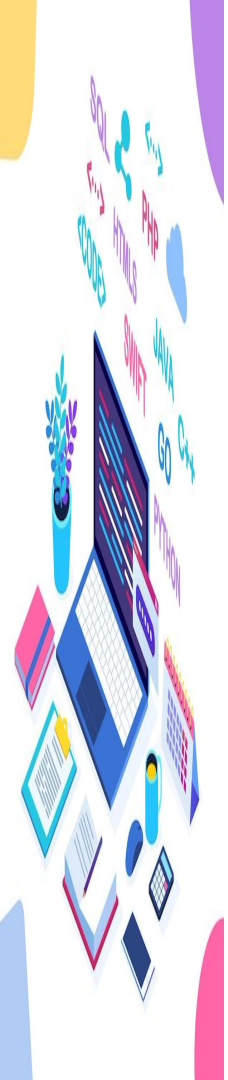
Java is used in various domains, including

- desktop applications,
- web development (Java Servlets, JavaServer Pages),
- mobile development (Android apps),
- server-side applications (Java EE, Spring framework),
- scientific and financial applications, and more.



HISTORY OF JAVA

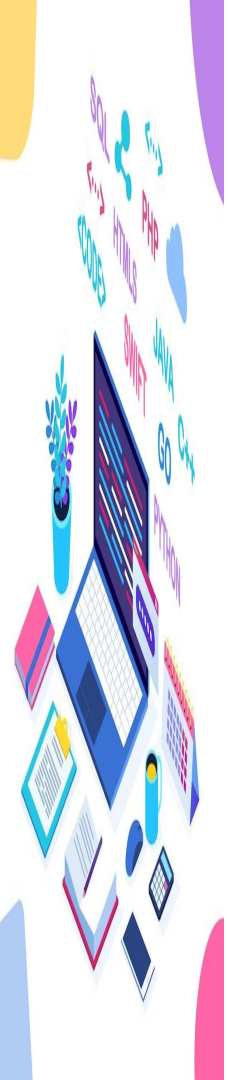
- Java is a widely-used programming language that was developed by Sun Microsystems (now owned by Oracle Corporation) in the mid-1990s.
- Java was the brainchild of:
  - James Gosling
  - Patrick Naughton
  - Chris Warth
  - EdFrank &
  - Frank Sheridan
- The origin of Java can be traced back to the fall of 1992, and was initially called Oak
- Oak was renamed as Java in 1995



HISTORY OF JAVA

Design Goal

- Java was originally meant to be a platform-neutral language for embedded software in devices
- The goal was to move away from platform and OS-specific compilers that would compile source for a particular target platform to a language that would be portable, and platform-independent
- The language could be used to produce platform-neutral code



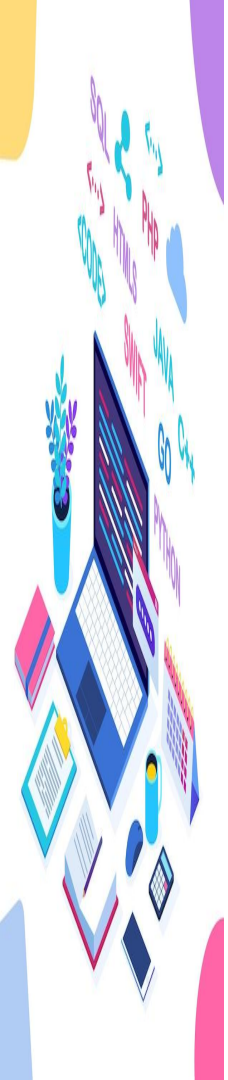
OVERVIEW OF JAVA

Java is a widely-used programming language that was developed by Sun Microsystems (now owned by Oracle Corporation) in the mid-1990s.

It is designed to be platform-independent, meaning that Java programs can run on any system that has a Java Virtual Machine (JVM) installed.

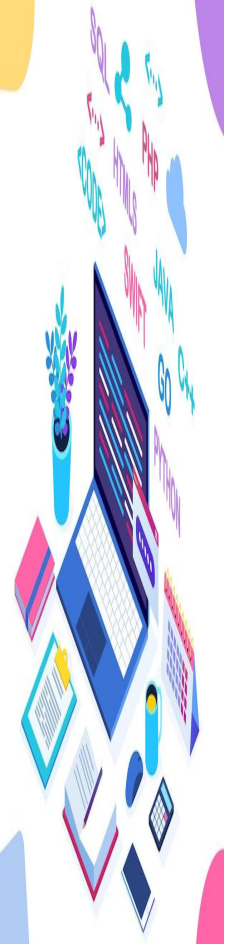
Java Virtual Machine (JVM): The JVM is a crucial component of the Java platform.

It provides an execution environment that interprets and executes Java bytecode. The JVM also includes various optimizations, such as just-in-time (JIT) compilation, which improves performance.



The Java “White Paper” Buzzwords

1. Simple
2. Object-Oriented
3. Distributed
4. Robust
5. Secure
6. Architecture-Neutral
7. Portable
8. Interpreted
9. High-Performance
10. Multithreaded
11. Dynamic

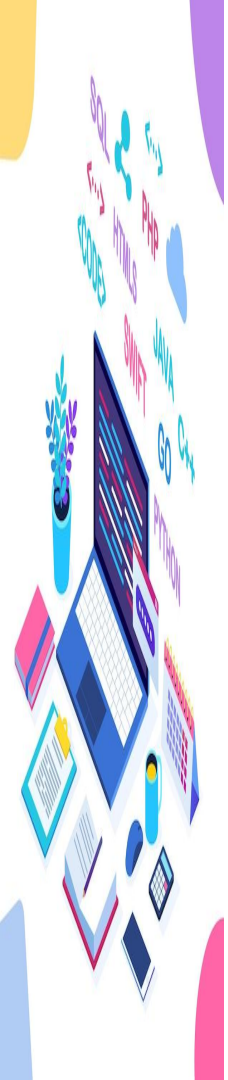


The Java “White Paper” Buzzwords

1. Simple

The syntax for Java is, indeed, a cleaned-up version of C++ syntax. There is no need for header files, pointer arithmetic (or even a pointer syntax), structures, unions, operator overloading, virtual base classes, and so on.

One of the goals of Java is to enable the construction of software that can run stand-alone on small machines. The size of the basic interpreter and class support is about 40K; the basic standard libraries and thread support.



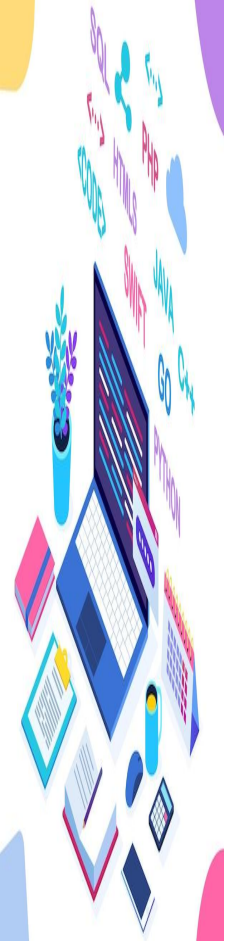
The Java “White Paper” Buzzwords

2. Object-Oriented

- The object-oriented design is a programming technique that focuses on the data—objects—and on the interfaces to those objects.

3. Distributed

- Java has an extensive library of routines for coping with TCP/IP protocols like HTTP and FTP. Java applications can open and access objects across the Net via URLs with the same ease as when accessing a local file system



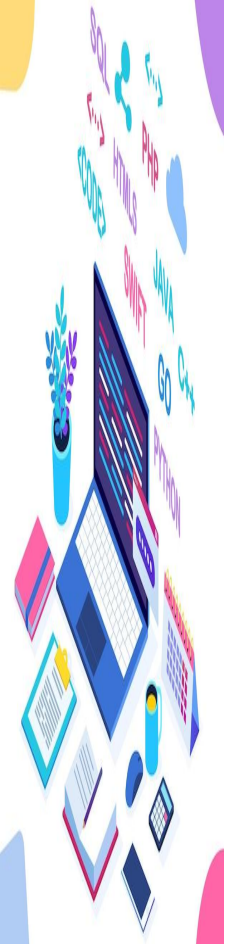
The Java “White Paper” Buzzwords

4. Robust

- Java provides early checking for possible problems, dynamic (runtime) checking, and eliminating situations that are error-prone. Java has a pointer model that eliminates the possibility of overwriting memory and corrupting data.

5. Secure

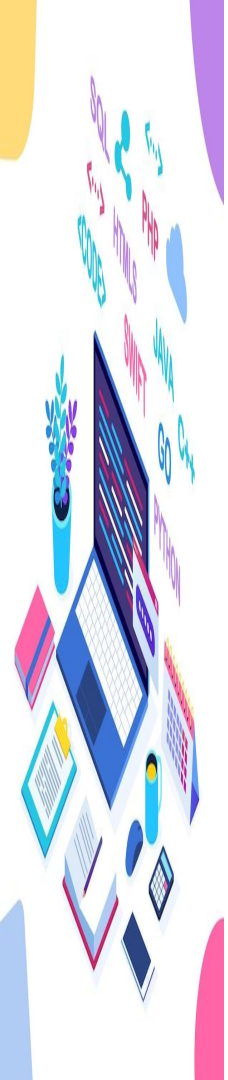
- Java is intended to be used in networked/distributed environments. Java enables the construction of virus-free, tamper-free systems. From the beginning, Java was designed to make certain kinds of attacks impossible, among them:
  - Overrunning the runtime stack—a common attack of worms and viruses
  - Corrupting memory outside its own process space
  - Reading or writing files without permission



The Java “White Paper” Buzzwords

6. Architecture-Neutral

- The compiler generates an architecture-neutral object file format. The compiled code is executable on many processors, given the presence of the Java runtime system. The Java compiler does this by generating bytecode instructions which have nothing to do with a particular computer architecture. Rather, they are designed to be both easy to interpret on any machine and easy to translate into native machine code on the fly.



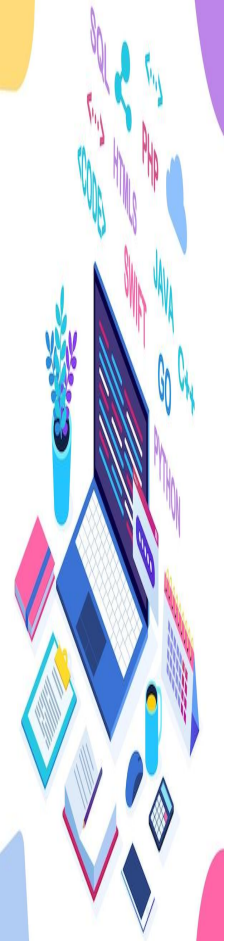
The Java “White Paper” Buzzwords

7. Portable

- Unlike C and C++, there are no “implementation-dependent” aspects of the specification. The sizes of the primitive data types are specified, as is the behavior of arithmetic on them.
- For example, an int in Java is always a 32-bit integer. In C/C++, int can mean a 16-bit integer, a 32-bit integer, or any other size that the compiler vendor likes.

8. Interpreted

- The Java interpreter can execute Java bytecodes directly on any machine to which the interpreter has been ported. Since linking is a more incremental and lightweight process, the development process can be much more rapid and exploratory



The Java “White Paper” Buzzwords

9. High-Performance

- While the performance of interpreted bytecodes is usually more than adequate, there are situations where higher performance is required. The bytecodes can be translated on the fly (at runtime) into machine code for the particular CPU the application is running on

10. Multithreaded

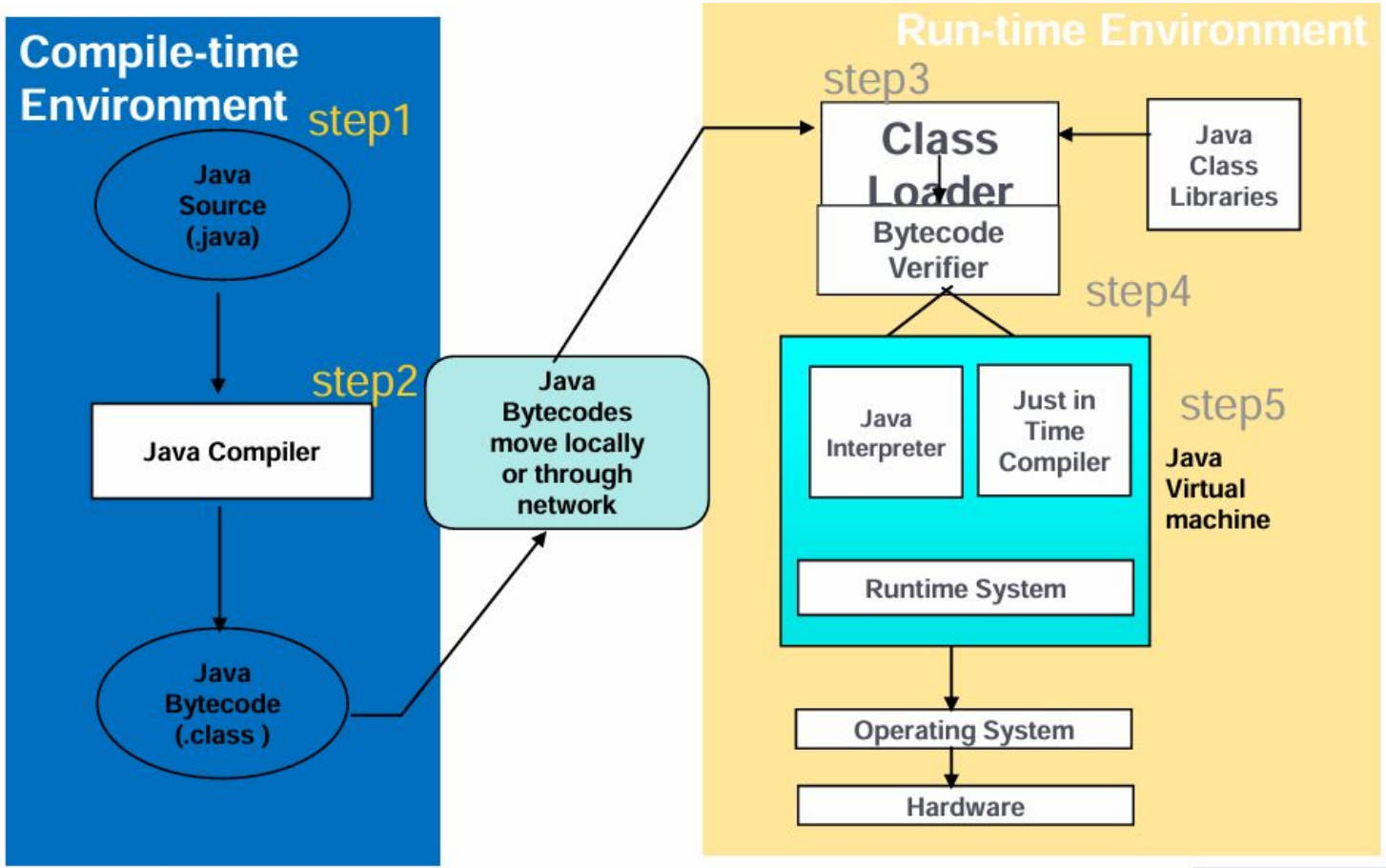
- The benefits of multithreading are better interactive responsiveness and real-time behavior.

11. Dynamic

- In a number of ways, Java is a more dynamic language than C or C++. It was designed to adapt to an evolving environment. Libraries can freely add new methods and instance variables without any effect on their clients.

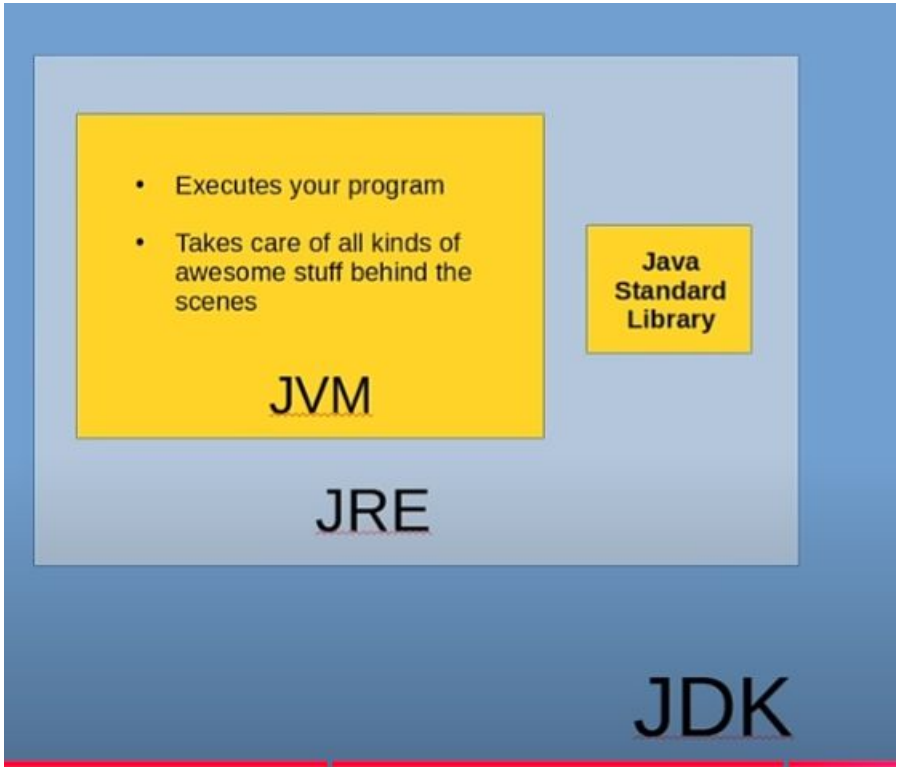


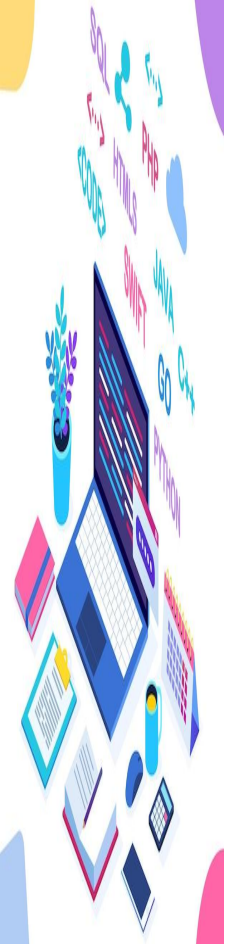
JAVA ARCHITECTURE





JAVA ARCHITECTURE





JAVA ARCHITECTURE

Step1:

- Create a java source code with .java extension

Step2:

- Compile the source code using java compiler, which will create bytecode file with .class extension

Step3:

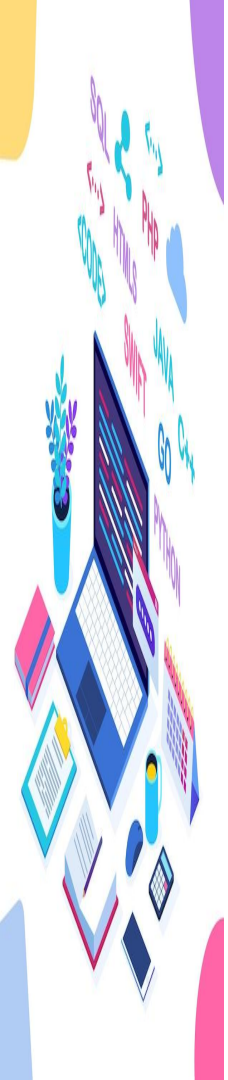
- Class loader reads both the user defined and library classes into the memory for execution

Step4:

- Bytecode verifier validates all the bytecodes are valid and do not violate Java's security restrictions

Step5:

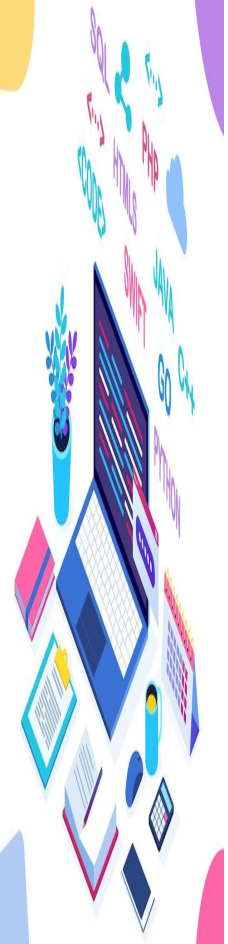
- JVM reads bytecodes and translates into machine code for execution. While execution of the program the code will interact to the operating system and hardware



JAVA ARCHITECTURE

- Java has long been a popular choice for developing enterprise-level software. Its scalability, performance, and extensive ecosystem of frameworks and tools have made it a preferred language for building large-scale applications, web services, and enterprise systems. Java Architecture is a collection of components, i.e., JVM, JRE, and JDK.

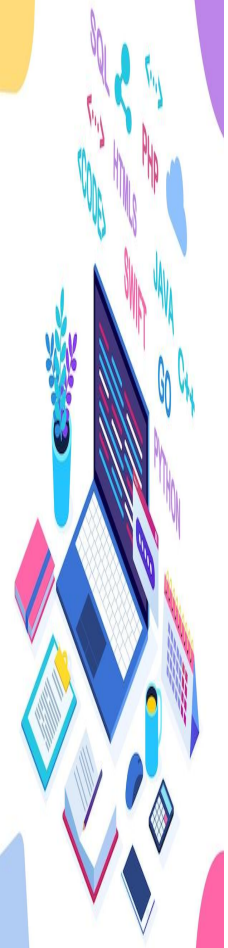
- It integrates the process of interpretation and compilation. It defines all the processes involved in creating a Java program



JAVA ARCHITECTURE

Java Development Kit (JDK):

- This is the most comprehensive component, providing the tools necessary for developing Java applications. It includes:
- **Java Runtime Environment (JRE):** This provides the necessary environment to run Java applications. It contains the JVM and core class libraries.
- **Development Tools:** These include the Java compiler (javac), Java interpreter(java), debugger (jdb), archiver (jar), and other utilities for creating, compiling, and packaging Java programs.



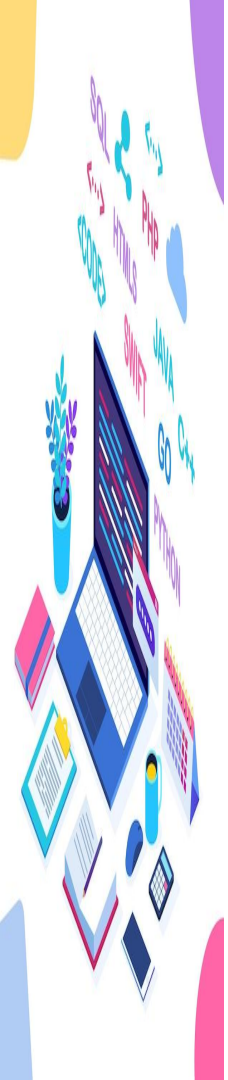
JAVA ARCHITECTURE

Java Runtime Environment (JRE):

- This is a subset of the JDK, specifically designed for running Java applications. It includes:

Java Virtual Machine (JVM): This is the core of Java's platform independence. It is an abstract machine that provides a runtime environment for executing Java bytecode. The JVM interprets and executes the bytecode, translating it into machine-specific instructions for the underlying hardware.

Java Class Libraries: These are a vast collection of pre-written classes and interfaces that provide common functionalities, such as I/O operations, networking, GUI development, and data structures.

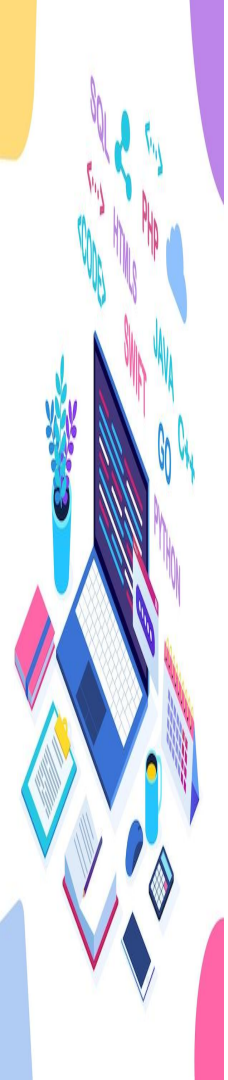


JAVA ARCHITECTURE

Java Class Libraries:

In the Java API, classes and interfaces are packaged in packages. The commonly used packages are

- **a.lang package** -A collection of classes and methods which includes the basic features such as string, math, exception and throwable
- **b.util package** - A collection of classes and methods for providing list, date and time.
- **c.sql package**-A collection of classes and methods for interacting with databases.
- **d.abstract window toolkit package** - A collection of classes and methods for implementing graphical user interface.
- **e.input/output package** - A collection of classes and methods for input output operations.



JAVA ARCHITECTURE

Java Virtual Machine (JVM):

The JVM is responsible for executing Java bytecode. Its key components include:

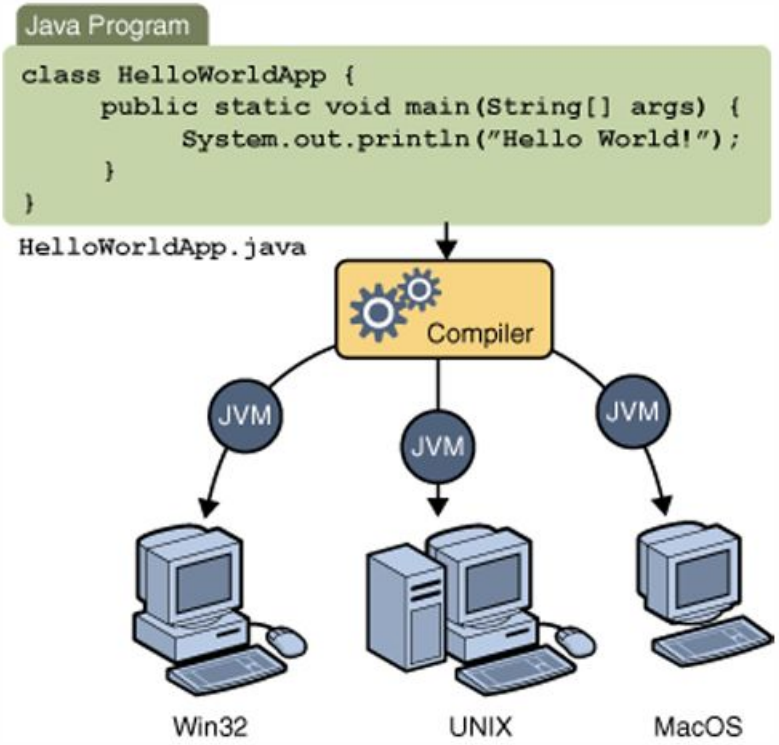
Class Loader: Loads .class files into memory.

Runtime Data Areas: These are memory areas used by the JVM during program execution, including the Method Area, Heap, Stack, PC Registers, and Native Method Stacks.

Execution Engine: This includes the interpreter, which executes bytecode instruction by instruction, and the Just-In-Time (JIT) compiler, which compiles frequently executed bytecode into native machine code for performance optimization.

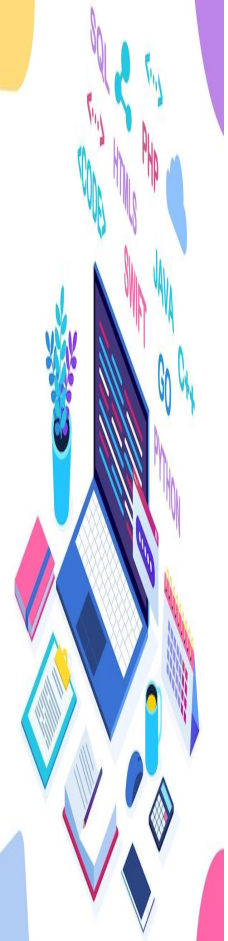
Garbage Collector: Manages memory by automatically reclaiming memory occupied by objects that are no longer referenced.

JAVA ARCHITECTURE



JAVA VIRTUAL MACHINE

- The output of the compiler is bytecode
- The bytecodes are executed by JVM
- It is an interpreter which converts the byte code to machine specific instructions and execute
- JVM is platform specific



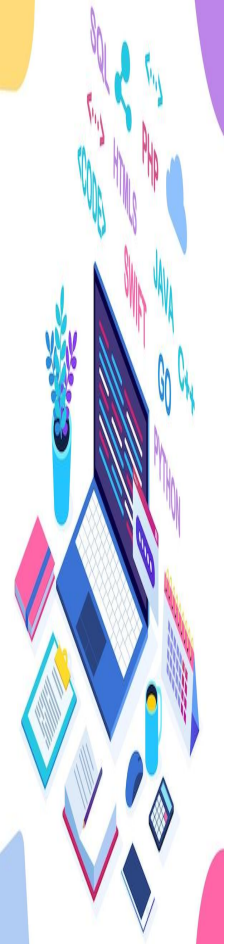
Data Types

- Java is a strongly typed language - variable has a type, every expression has a type, and every type is strictly defined.
- All assignments, whether explicit or via parameter passing in method calls, are checked for type compatibility.
- The Java compiler checks all expressions and parameters to ensure that the types are compatible. Any type mismatches are errors that must be corrected before the compiler will finish compiling the class.

A data type in Java defines the type of data a variable can hold. It specifies

  - What kind of values a variable can store
  - The operations that can be performed on it
  - How much memory is required to store the value.

Data types help Java to understand the kind of operations to perform on the stored values, such as arithmetic operations for numbers, logical comparisons for booleans, and manipulation for characters.



Data Types

The Primitive Types

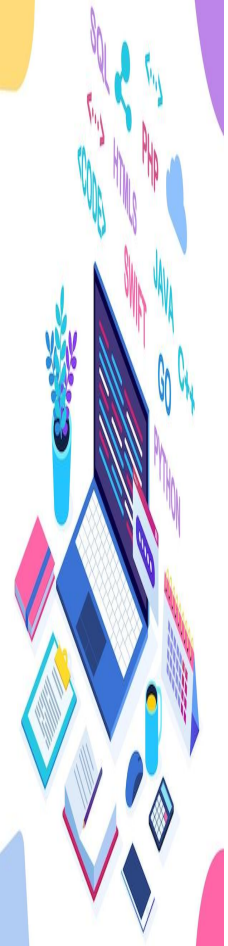
- **Integers** This group includes byte, short, int, and long, which are for whole-valued signed numbers.
- **Floating-point numbers** This group includes float and double, which represent numbers with fractional precision.
- **Characters** This group includes char, which represents symbols in a character set, like letters and numbers.
- **Boolean** This group includes boolean, which is a special type for representing true/false values.



Data Types

Integers

Name	Width	Range
long	64	–9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
int	32	–2,147,483,648 to 2,147,483,647
short	16	–32,768 to 32,767
byte	8	–128 to 127

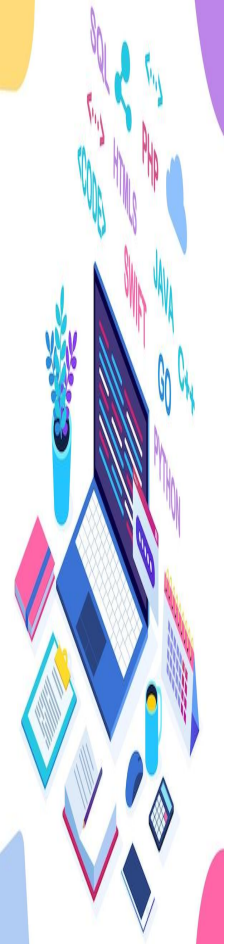


Data Types

Integers

```
class Light {
  public static void main(String args[]) {
    int lightspeed;
    long days;
    long seconds;
    long distance;
    // approximate speed of light in miles per second
    lightspeed = 186000;
    days = 1000; // specify number of days here
    seconds = days * 24 * 60 * 60; // convert to seconds
    distance = lightspeed * seconds; // compute distance
    System.out.print("In " + days);
    System.out.print(" days light will travel about ");
    System.out.println(distance + " miles.");
  }
}
```

In 1000 days light will travel about 160704000000000 miles.

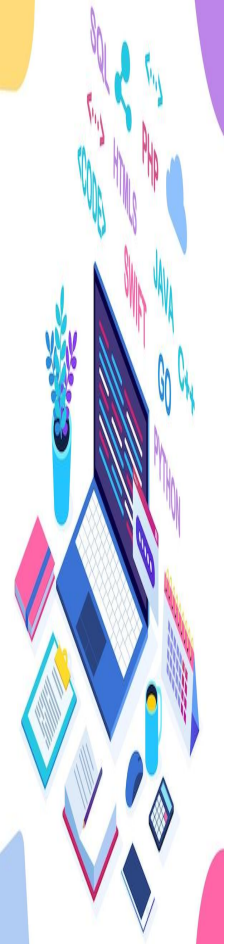


Data Types

Floating-point numbers

```
 // Compute the area of a circle.
class Area {
  public static void main(String args[]) {
    double pi, r, a;
    r = 10.8; // radius of circle
    pi = 3.1416; // pi, approximately
  a = pi * r * r; // compute area
   System.out.println("Area of circle is " + a);
  }
}
```

Name	Width in Bits	Approximate Range
double	64	4.9e–324 to 1.8e+308
float	32	1.4e–045 to 3.4e+038



Data Types

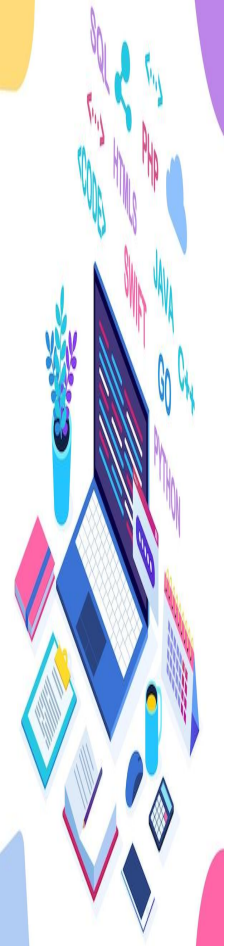
Characters

In Java char is a 16-bit type.

The range of a char is 0 to 65,536. There are no negative chars. The standard set of characters known as ASCII still ranges from 0 to 127.

ASCII Table

Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char
0	0	0		32	20	40	[space]	64	40	100	@	96	60	140	`
1	1	1		33	21	41	!	65	41	101	A	97	61	141	a
2	2	2		34	22	42	"	66	42	102	B	98	62	142	b
3	3	3		35	23	43	#	67	43	103	C	99	63	143	c
4	4	4		36	24	44	$	68	44	104	D	100	64	144	d
5	5	5		37	25	45	%	69	45	105	E	101	65	145	e
6	6	6		38	26	46	&	70	46	106	F	102	66	146	f
7	7	7		39	27	47	'	71	47	107	G	103	67	147	g
8	8	10		40	28	50	(	72	48	110	H	104	68	150	h
9	9	11		41	29	51	)	73	49	111	I	105	69	151	i
10	A	12		42	2A	52	*	74	4A	112	J	106	6A	152	j
11	B	13		43	2B	53	+	75	4B	113	K	107	6B	153	k
12	C	14		44	2C	54	,	76	4C	114	L	108	6C	154	l
13	D	15		45	2D	55	-	77	4D	115	M	109	6D	155	m
14	E	16		46	2E	56	.	78	4E	116	N	110	6E	156	n
15	F	17		47	2F	57	/	79	4F	117	O	111	6F	157	o
16	10	20		48	30	60	0	80	50	120	P	112	70	160	p
17	11	21		49	31	61	1	81	51	121	Q	113	71	161	q
18	12	22		50	32	62	2	82	52	122	R	114	72	162	r
19	13	23		51	33	63	3	83	53	123	S	115	73	163	s
20	14	24		52	34	64	4	84	54	124	T	116	74	164	t
21	15	25		53	35	65	5	85	55	125	U	117	75	165	u
22	16	26		54	36	66	6	86	56	126	V	118	76	166	v
23	17	27		55	37	67	7	87	57	127	W	119	77	167	w
24	18	30		56	38	70	8	88	58	130	X	120	78	170	x
25	19	31		57	39	71	9	89	59	131	Y	121	79	171	y
26	1A	32		58	3A	72	:	90	5A	132	Z	122	7A	172	z
27	1B	33		59	3B	73	;	91	5B	133	[	123	7B	173	{
28	1C	34		60	3C	74	<	92	5C	134	\	124	7C	174	|
29	1D	35		61	3D	75	=	93	5D	135	]	125	7D	175	}
30	1E	36		62	3E	76	>	94	5E	136	^	126	7E	176	~
31	1F	37		63	3F	77	?	95	5F	137	_	127	7F	177	



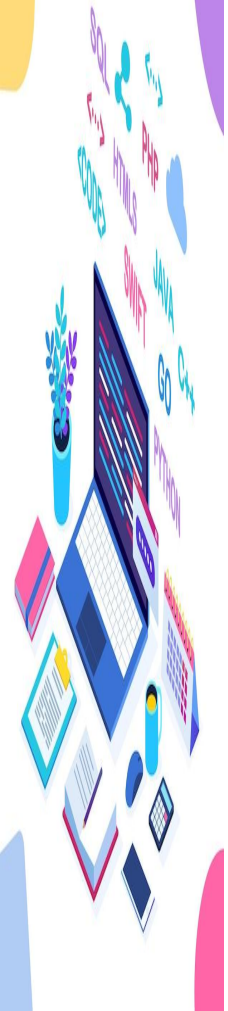
Data Types

Characters

```
// Demonstrate char data type.
class CharDemo {
  public static void main(String args[]) {
    char ch1, ch2;
    ch1 = 88; // code for X
    ch2 = 'Y';
    System.out.print("ch1 and ch2: ");
    System.out.println(ch1 + " " + ch2);
  }
}
```

This program displays the following output:

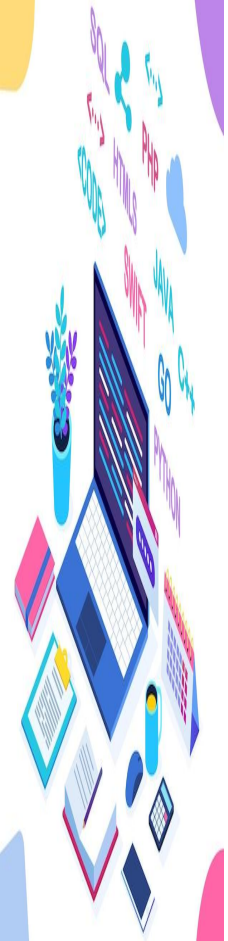
```
ch1 and ch2: X Y
```



Data Types

Booleans

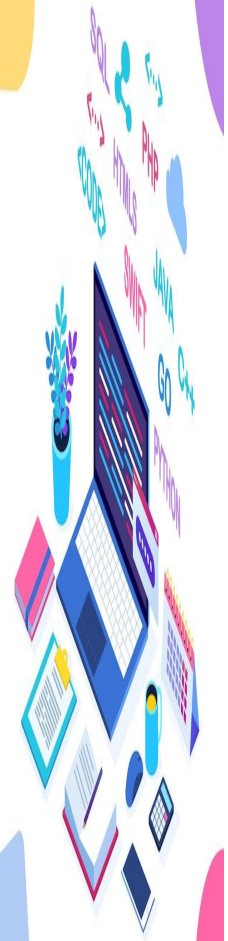
- Java has a primitive type, called boolean, for logical values. It can have only one of two possible values, true or false.
- This is the type returned by all relational operators, as in the case of a<b.
- boolean is also the type required by the conditional expressions that govern the control statements such as if and for.



Data Types

Java's Automatic Conversions

- When one type of data is assigned to another type of variable, an automatic type conversion will take place if the following two conditions are met:
  - The two types are compatible.
  - The destination type is larger than the source type.
- When these two conditions are met, a widening conversion takes place. For example, the int type is always large enough to hold all valid byte values, so no explicit cast statement is required.
- For widening conversions, the numeric types, including integer and floating-point types, are compatible with each other.
- However, there are no automatic conversions from the numeric types to char or boolean. Also, char and boolean are not compatible with each other.

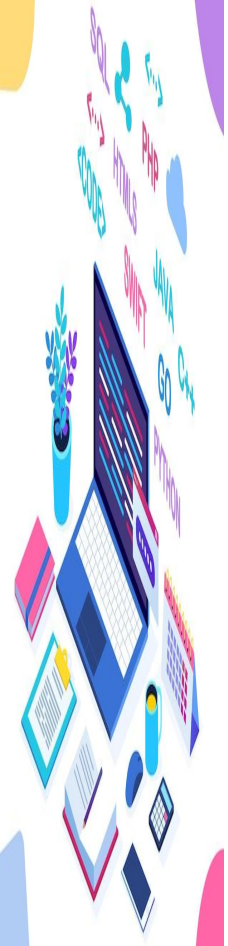


Data Types

Casting Incompatible Types

- If you want to assign an int value to a byte variable? This conversion will not be performed automatically, because a byte is smaller than an int. This kind of conversion is sometimes called a narrowing conversion, since you are explicitly making the value narrower so that it will fit into the target type.
- To create a conversion between two incompatible types, you must use a cast. A cast is simply an explicit type conversion. It has this general form:

  (target-type) value
- The target-type specifies the desired type to convert the specified value to. For example, the following fragment casts an int to a byte. If the integer's value is larger than the range of a byte, it will be reduced modulo (the remainder of an integer division by the) byte's range.
- A different type of conversion will occur when a floating-point value is assigned to an integer type: truncation.



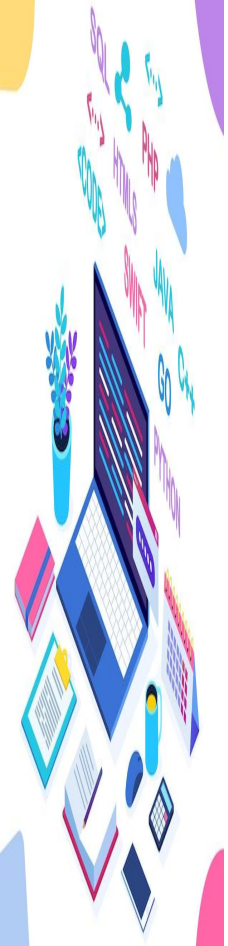
Data Types

Casting Incompatible Types

```
class Conversion {
  public static void main(String args[]) {
    byte b;
    int i = 257;
    double d = 323.142;
    System.out.println("\nConversion of int to byte.");
    b = (byte) i;
    System.out.println("i and b " + i + " " + b);
    System.out.println("\nConversion of double to int.");
    i = (int) d;
    System.out.println("d and i " + d + " " + i);
    System.out.println("\nConversion of double to byte.");
    b = (byte) d;
    System.out.println("d and b " + d + " " + b);
  }
}
```

This program generates the following output:

```
Conversion of int to byte.
i and b 257 1
Conversion of double to int.
d and i 323.142 323
Conversion of double to byte.
d and b 323.142 67
```

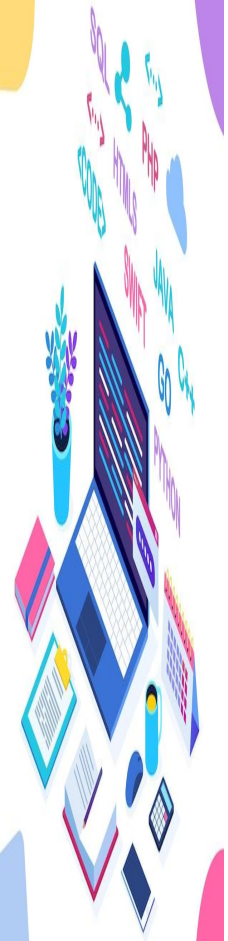


Data Types

Casting Incompatible Types

```
class Promote {
 public static void main(String args[]) {
   byte b = 42;
   char c = 'a';
   short s = 1024;
   int i = 50000;
   float f = 5.67f;
   double d = .1234;
   double result = (f * b) + (i / c) - (d * s);
   System.out.println((f * b) + " + " + (i / c) + " - " + (d * s));
   System.out.println("result = " + result);
 }
}
```

238.14 + 515 - 126.3616
result = 626.7784146484375



Variables

Variables

The variable is the basic unit of storage in a Java program. A variable is defined by the combination of an identifier, a type, and an optional initializer. In addition, all variables have a scope, which defines their visibility, and a lifetime.

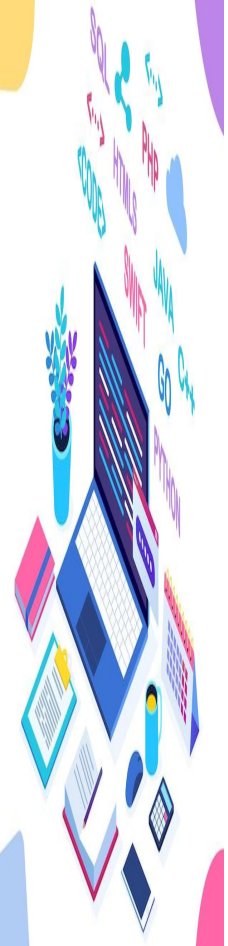
Declaring a Variable

In Java, all variables must be declared before they can be used. The basic form of a variable declaration is shown here:

type identifier [= value][, identifier [= value] …];

Here, type is one of Java's atomic types, or the name of a class or interface.

- Begin each variable with a lowercase letter
- Subsequent words should be capitalized:
  - **myVariable**
- Names are case-sensitive −Names can't include white space



Input and Output

OUTPUT METHODS

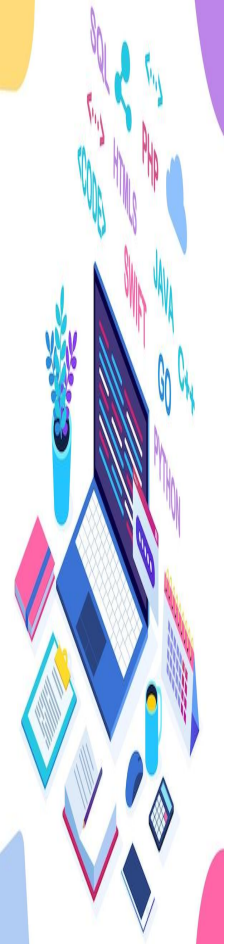
System.out.print():

- Syntax: System.out.print(argument);
- Purpose: Prints the argument to the console without adding a newline at the end. This means subsequent printing will continue on the same line.
- Argument: Can be any type, including String, int, double, etc. If the argument is not a String, Java converts it to a String using the String.valueOf(argument) method.
- If no argument given ,its an error.

```
public class test
{ // Main method
public static void main(String[]
args)
{
System.out.print("Hi Java");
System.out.print("Hi Java");
System.out.print("Hi Java");
}
}
```

Output

Hi Java Hi Java Hi Java



Input and Output

OUTPUT METHODS

System.out.println():

• Syntax: System.out.println(argument);

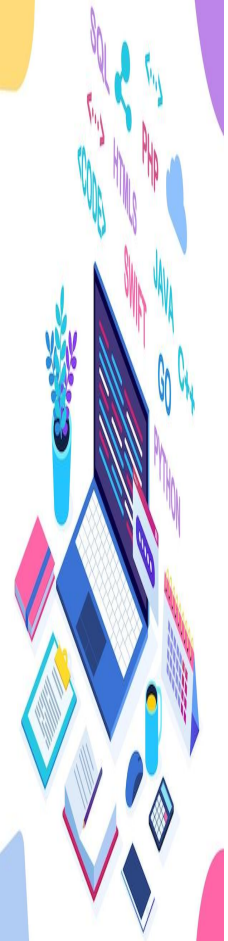
• Purpose: Prints the argument to the console and adds a newline at the end. This means subsequent printing will start from the next line.

• Argument: Can be any type, similar to System.out.print. If no argument is provided, it simply prints a newline.

```
public class test
{ // Main method
public static void main(String[]
args)
{
System.out.println("Hi Java");
System.out.println("Hi Java");
System.out.println("Hi Java");
}
}
```

Output

```
Hi Java
Hi Java
Hi Java
```



Input and Output

OUTPUT METHODS

System.out.printf():

The System.out.printf method in Java is used for formatted output, allowing you to print formatted strings directly to the console. It is part of the System.out object, which is an instance of PrintStream.

Syntax: System.out.printf(String format, Object... args);

Parameters:

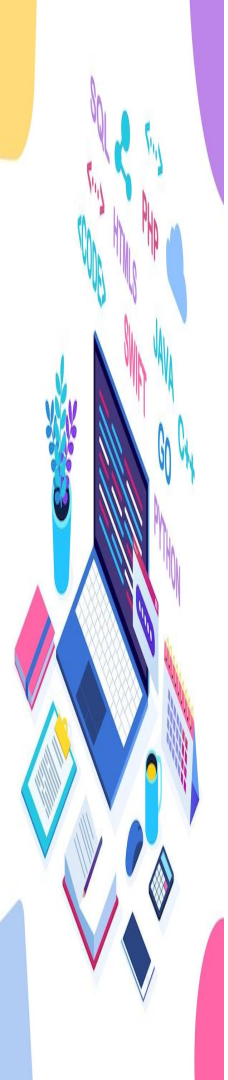
format: A format string containing text and format specifiers that dictate how the arguments should be formatted.

args: A list of arguments that correspond to the format specifiers in the format string

```
public class test
{ // Main method
public static void main(String[]
args)
{
System.out.printf("Java is
number %d language",1);
}
}
```

Output

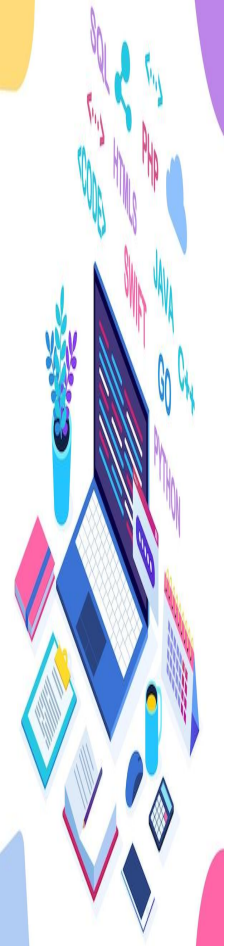
Java is number 1 language



Input

- The Scanner class in Java is located within the java.util library.
- To collect input from the keyboard in Java, there are several methods available, with the java.util.Scanner being one of the notable ones.
- Commonly utilized for breaking down text into strings and various basic data types, the Java Scanner class leverages regular expressions to accomplish this.
- Acquiring user input in Java is most straightforward when using the Scanner class.
- Java's Scanner allows the acquisition of user data in fundamental data types like integers, long integers, doubles, bytes, floats, shorts, and more.
- **Step 1:Create Instance for Scanner class**

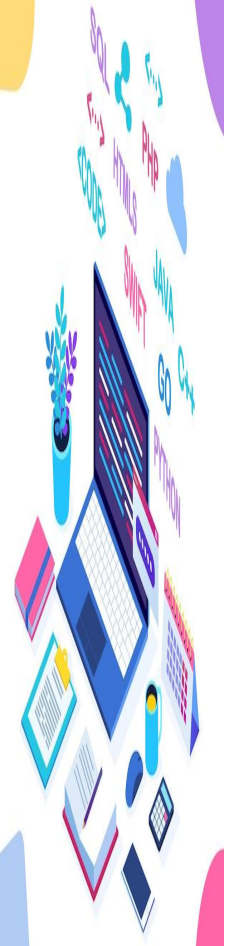
```
Scanner in = new Scanner(System.in);
```



Input

Step 2: To get Inputs of different datatypes-use the scanner object and invoke the following methods

String	next()	It is used to get the next complete token from the scanner which is in use.
BigDecimal	nextBigDecimal()	It scans the next token of the input as a BigDecimal.
BigInteger	nextBigInteger()	It scans the next token of the input as a BigInteger.
boolean	nextBoolean()	It scans the next token of the input into a boolean value and returns that value.
byte	nextByte()	It scans the next token of the input as a byte.
double	nextDouble()	It scans the next token of the input as a double.
float	nextFloat()	It scans the next token of the input as a float.
int	nextInt()	It scans the next token of the input as an Int.
String	nextLine()	It is used to get the input string that was skipped of the Scanner object.
long	nextLong()	It scans the next token of the input as a long.



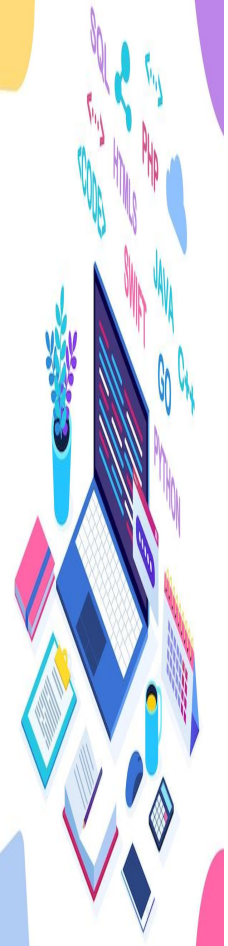
GETTING INPUT FROM KEYBOARD

```
import java.util.Scanner;
public class InputExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in); // Create
Scanner object
        System.out.print("Enter your name: ");
        String name = sc.nextLine();  // Read String
input
        System.out.print("Enter your age: ");
        int age = sc.nextInt();       // Read integer input
        System.out.println("Name " + name);
          System.out.println("Name " + age);
    }
}
```

Output

Enter your name:Anu
Enter your Age:21

Name:Anu
Age:21



Operators

- Java provides a rich operator environment.
- Most of its operators can be divided into the following four groups
  - Arithmetic
  - Bitwise
  - Relational
  - Logical
- Java also defines some additional operators that handle certain special situations



Operators

Arithmetic Operators

- Arithmetic operators are used in mathematical expressions in the same way that they are used in algebra.

Operator	Result
+	Addition (also unary plus)
–	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus
++	Increment
+=	Addition assignment
– =	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Modulus assignment
– –	Decrement



Operators

Arithmetic Operators

- Arithmetic operators are used in mathematical expressions in the same way that they are used in algebra. The operands of the arithmetic operators must be of a numeric type. You cannot use them on boolean types, but you can use them on char types, since the char type in Java is, essentially, a subset of int.

Operator	Result
+	Addition (also unary plus)
–	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus
++	Increment
+=	Addition assignment
– =	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Modulus assignment
– –	Decrement

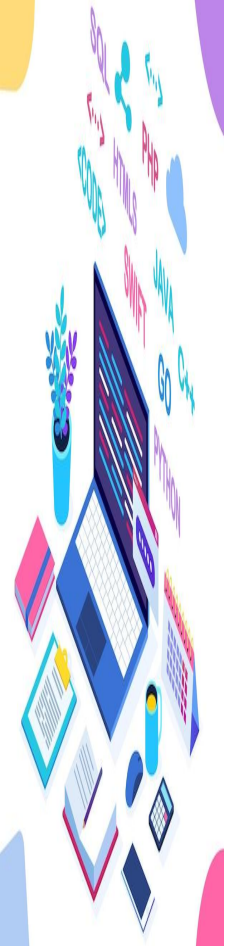


Operators

Relational Operators

- The relational operators determine the relationship that one operand has to the other.
- Specifically, they determine equality and ordering

Operator	Description	Example
==	Two values are checked, and if equal, then the condition becomes true	(A == B)
!=	Two values are checked to determine whether they are equal or not, and if not equal, then the condition becomes true	(A != B)
>	Two values are checked and if the value on the left is greater than the value on the right, then the condition becomes true.	(A > B)
<	Two values are checked and if the value on the left is less than the value on the right, then the condition becomes true	(A < B)
>=	Two values are checked and if the value on the left is greater than equal to the value on the right, then the condition becomes true	(A >= B)
<=	Two values are checked and if the value on the left is less than equal to the value on the right, then the condition becomes true	(A <= B)



Operators

Logical Operators: Logical operators are used in programming to connect two or more conditions or expressions, yielding a single Boolean value of true or false

- AND (&&): Java does not evaluate the second operand if the first is false, because the result will be false regardless of the value of the second operand.
- OR (||): Java does not evaluate the second operand if the first is true, because the result will be true regardless of the value of the second operand.
- This behavior, known as short-circuiting Logical operators in Java strictly work with boolean values. You cannot use them with numbers or objects directly without a condition that evaluates to a boolean.

Operator	Description	Example
&&	This is known as Logical AND & it combines two variables or expressions and if and only if both the operands are true, then it will return true	(A && B) is false
||	This is known as Logical OR & it combines two variables or expressions and if either one is true or both the operands are true, then it will return true	(A || B) is true
!	Called Logical NOT Operator. It reverses the value of a Boolean expression	!(A && B) is true



Operators

Bitwise Operators

- The & operator compares corresponding bits between two numbers and if both the bits are 1, only then the resultant bit is 1. If either one of the bits is 0, then the resultant bit is 0.

The Bitwise AND

- **Example :**
- int x = 7; int y = 9;

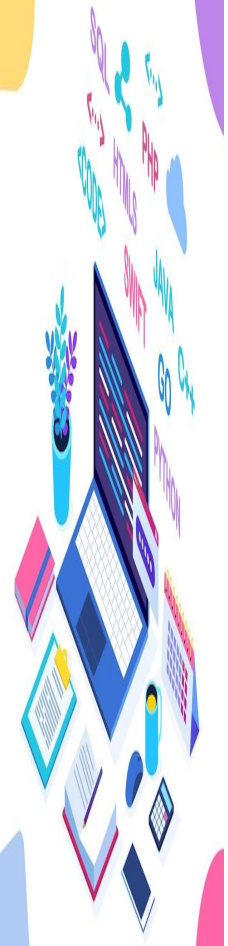
What will be x & y ?

42 00101010 &
15 00001111

00001010 -> 10

- x & y results in 10

Operator	Result
~	Bitwise unary NOT
&	Bitwise AND
|	Bitwise OR
^	Bitwise exclusive OR
>>	Shift right
>>>	Shift right zero fill
<<	Shift left
&=	Bitwise AND assignment
|=	Bitwise OR assignment
^=	Bitwise exclusive OR assignment
>>=	Shift right assignment
>>>=	Shift right zero fill assignment
<<=	Shift left assignment



Operators

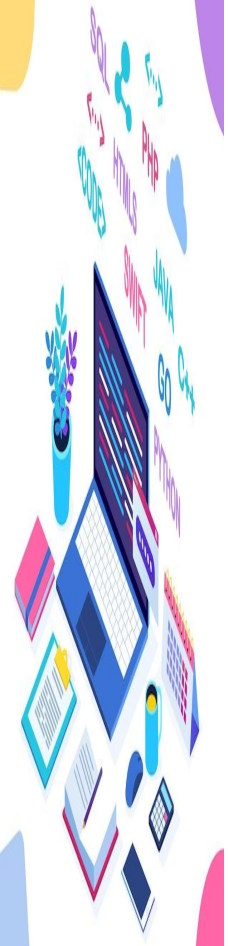
Bitwise Operators

The Bitwise OR

```
42  00101010  |
15  00001111
---------------
    00101111 -> 47
---------------
```

The Bitwise AND

```
42  00101010  |
15  00001111
---------------
    00100101 -> 37
---------------
```



Operators

Bitwise Operators - Left Shift

- The left shift operator, <<, shifts all of the bits in a value to the left a specified number of times. It has this general form:

  value << num

- Let us understand the use of left shift operator with the following example : int x = 8;

```
int y=4;
 x = x << y;
```

Answer=128

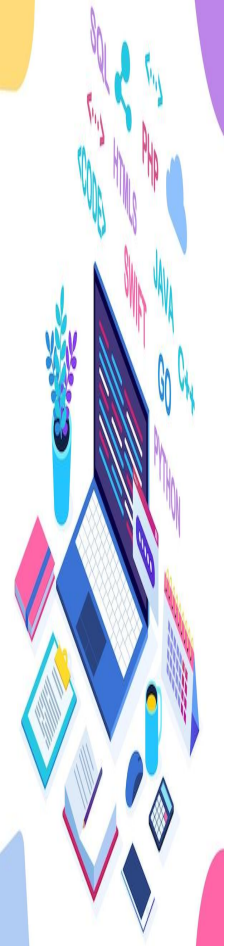
- **Example :**

When we represent 8 in binary form, we will get the following binary value :

0 0 0 0 1 0 0 0

When we apply >> which is the right shift operator, the bit represented by 1 moves by 3 positions to the right(represented by the number after the right shift operator). After shifting the binary digit, we will get :

1 0 0 0 0 0 0 0



Operators

Bitwise Operators - Right Shift

- Let us understand the use of right shift operator with the following example : int x = 16;

int y=3;

x = x >> y;

Answer=2

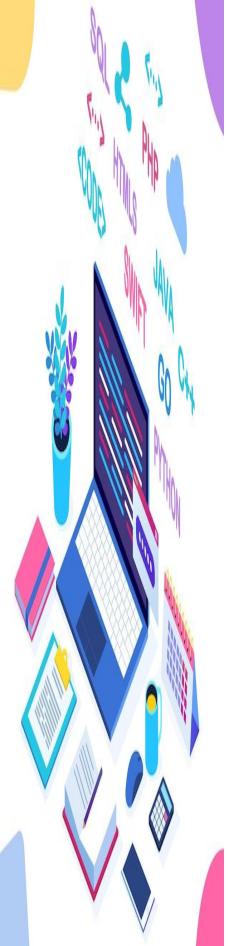
- **Example :**

When we represent 8 in binary form, we will get the following binary value :

0 0 0 1 0 0 0 0

When we apply >> which is the right shift operator, the bit represented by 1 moves by 3 positions to the right(represented by the number after the right shift operator). After shifting the binary digit 1, we will get :

0 0 0 0 0 0 1 0



Operators

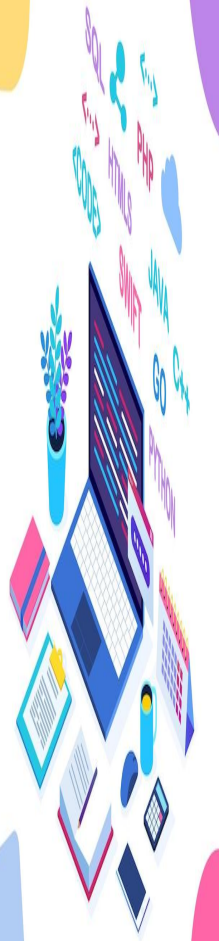
The ? Operator

Java includes a special ternary (three-way) operator that can replace certain types of if-then else statements. This operator is the ?. It can seem somewhat confusing at first, but the ? can be used very effectively once mastered. The ? has this general form:

expression1 ? expression2 : expression3

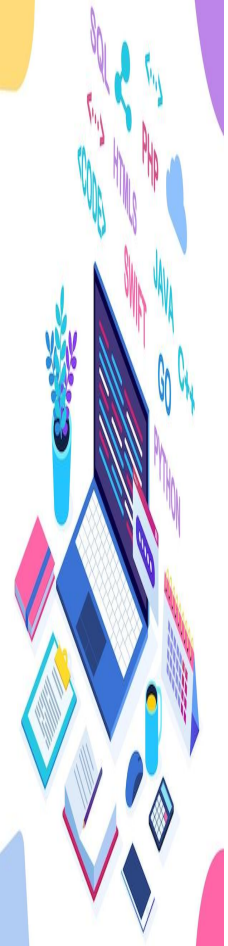
Here, expression1 can be any expression that evaluates to a Boolean value. If expression1 is true, then expression2 is evaluated; otherwise, expression3 is evaluated. Both expression2 and expression3 are required to return the same (or compatible) type, which can't be void.

```
int i = 10;
int k = i < 0 ? -i : i; // get absolute value of i
```



Operator Precedence

Highest						
++ (postfix)	– – (postfix)					
++ (prefix)	– – (prefix)	~	!	+ (unary)	– (unary)	(*type-cast*)
*	/	%				
+	–					
>>	>>>	<<				
>	>=	<	<=	instanceof		
==	!=					
&						
^						
|						
&&						
||						
?:						
–>						
=	op=					
Lowest						



Control Statements

Control statements are statements which alter the normal execution flow of a program

- There are three types of Control Statements in java

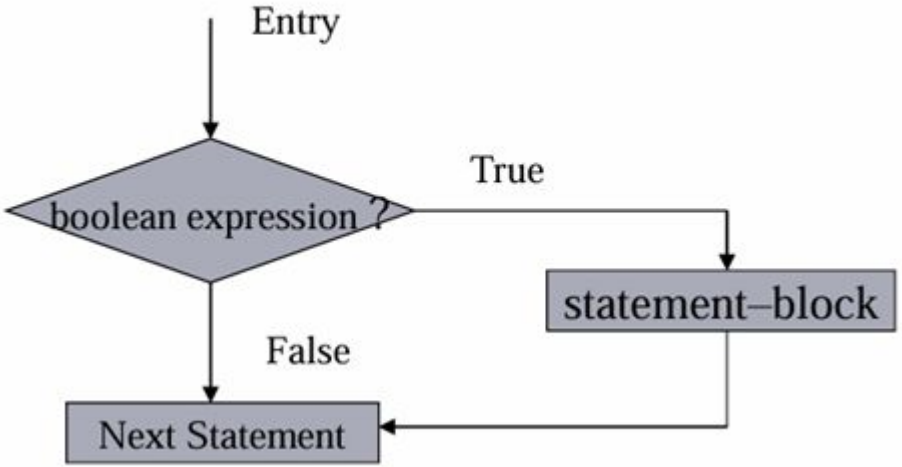
Selection statement	Iteration Statement	Jumping Statement
if	while	break
if – else	do – while	continue
switch	for	return

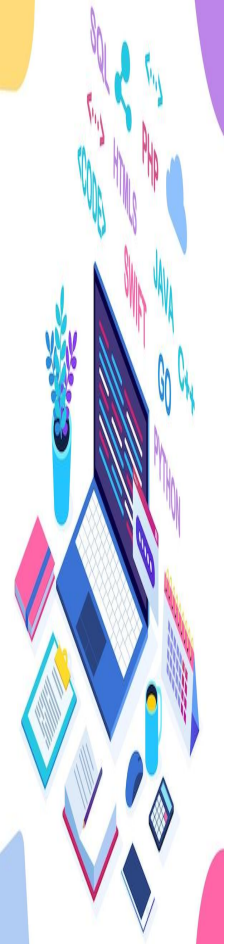


Simple if statement

Syntax :

```
if(boolean expression)
{
        statement–block;
}
Next statement;
```



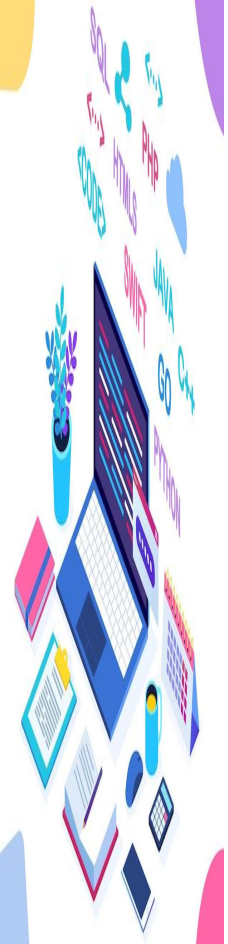


Simple if statement

Example:

```
if (yourSales >= target)
{
  performance = "Satisfactory";
  bonus = 100 + 0.01 * (yourSales - target);
}
```



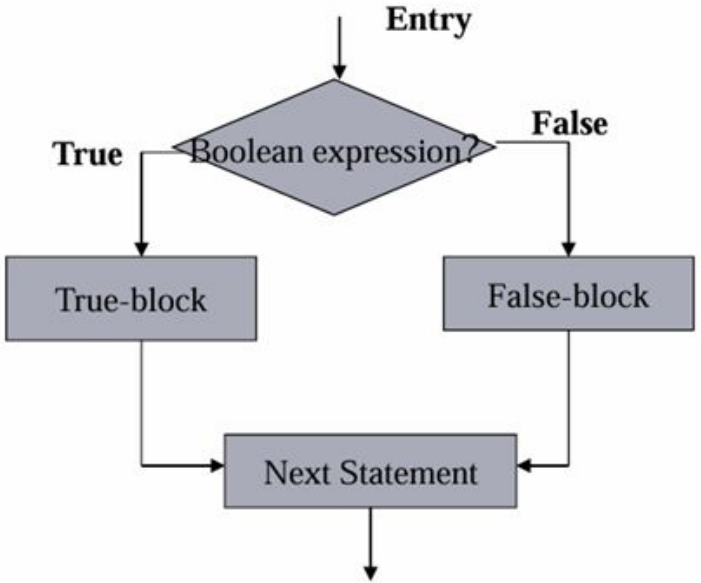


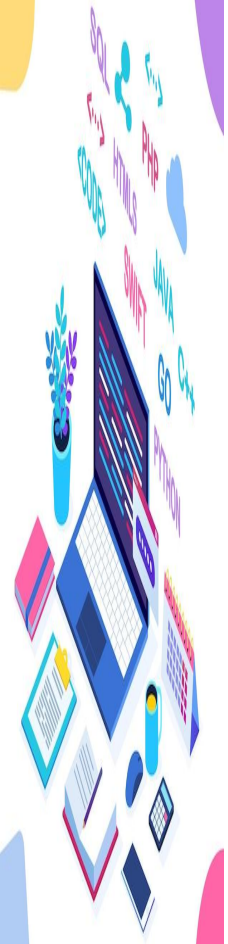
If..else statement

The if….else statement is an extension of simple if statement

Syntax :

```
if(boolean expression)
{
        True-block statements;
}
else
{
        False-block statements;
}
Next statement;
```

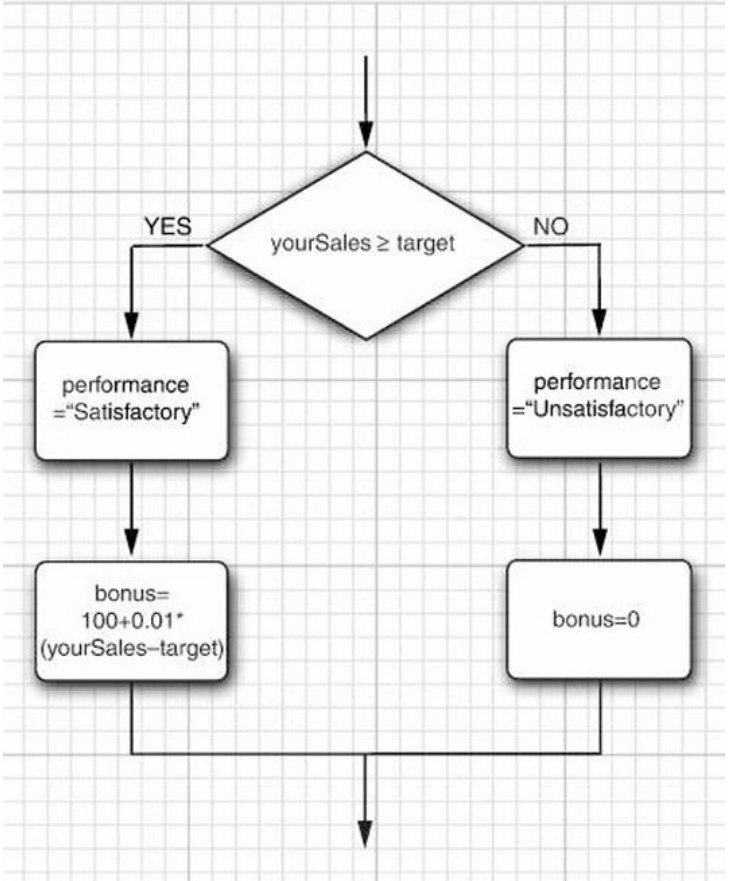




If..else statement

Example:

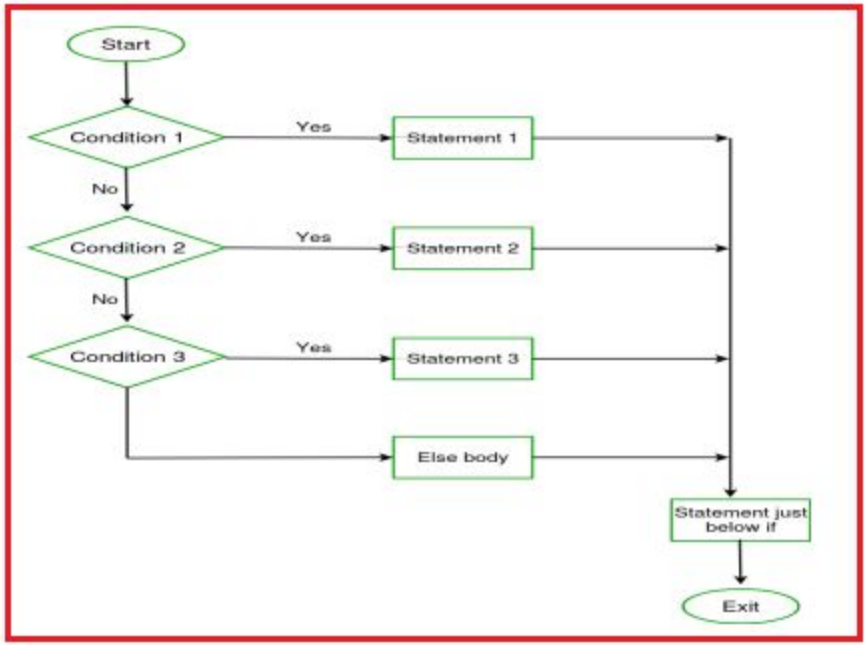
```
if (yourSales >= target)
{
  performance = "Satisfactory";
  bonus = 100 + 0.01 * (yourSales - target);
}
 else
{
  performance = "Unsatisfactory";
  bonus = 0;
}
```



Cascading if- else

Syntax :

```
if (condition1)
{
        statement-1;
 }
 ….
else if(condition)
{
        statement-n;
 }
 else
{
        default statement;
 }
 next statement;
```

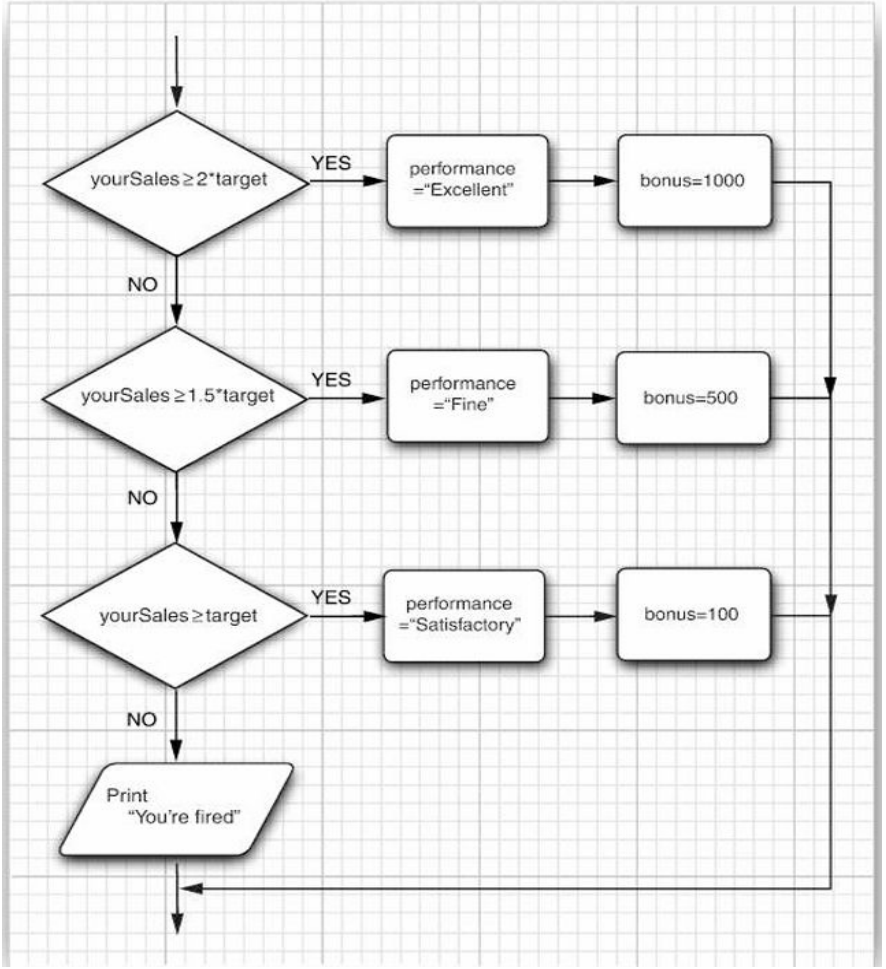


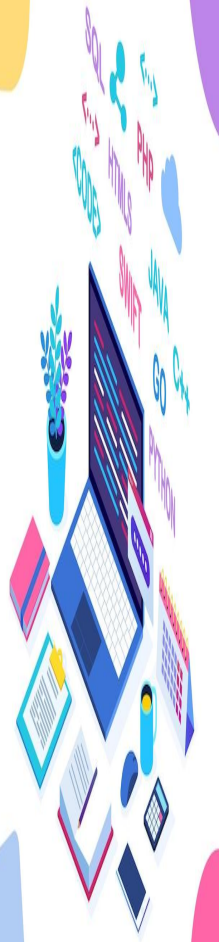


Cascading if- else

Example:

```
if (yourSales >= 2 * target)
{
  performance = "Excellent";
  bonus = 1000;
}
else if (yourSales >= 1.5 * target)
{
  performance = "Fine";
  bonus = 500;
}
else if (yourSales >= target)
{
  performance = "Satisfactory";
  bonus = 100;
}
else
{
        System.out.println("You're fired");
}
```



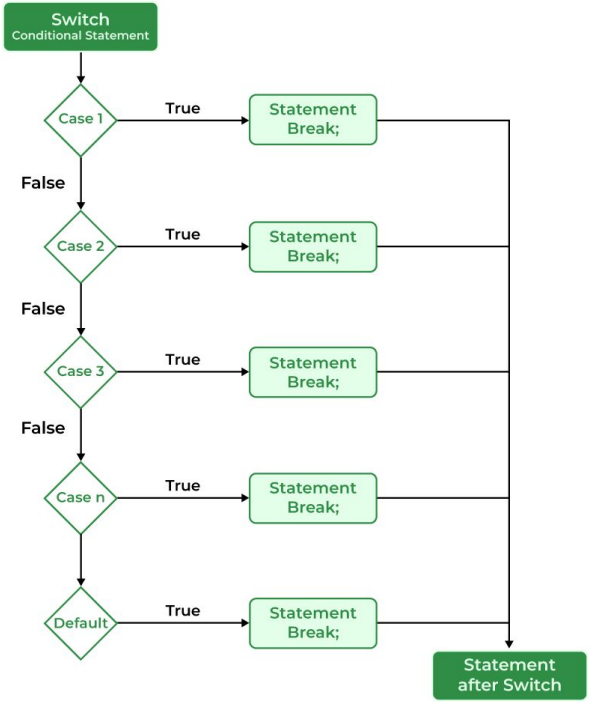


Switch case

The switch-case conditional construct is a more structured way of testing for multiple conditions rather than resorting to a multiple if statement

Syntax :

```
switch (expression)
{
case value-1 :
case-1 block
break;
case value-2 :
case-2 block
break;
default  :
default block
break;
}
statement-x;
```

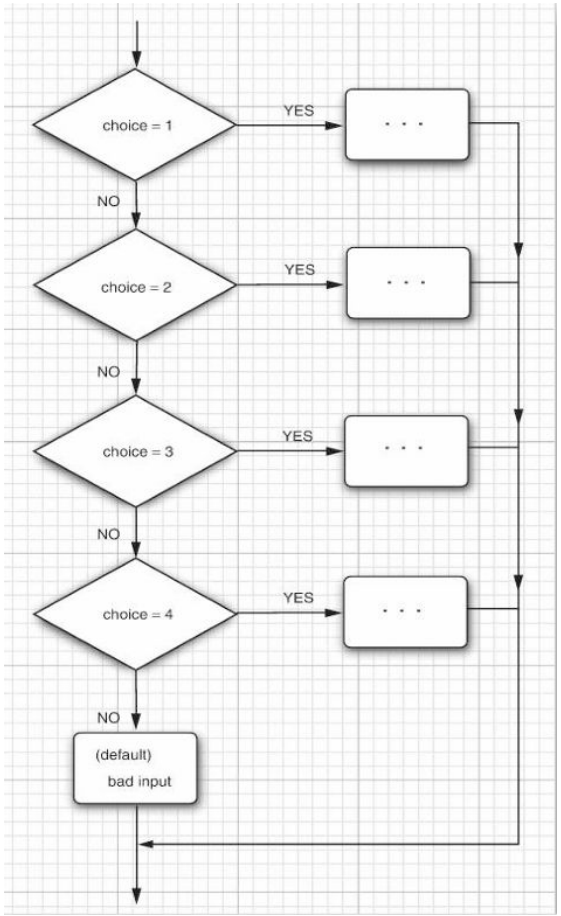


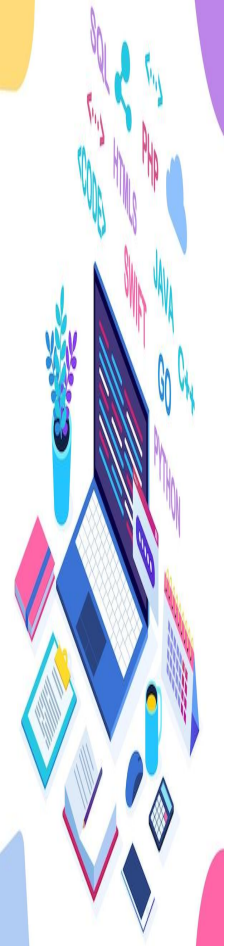


Switch case

Example:

```
Scanner in = new Scanner(System.in);
System.out.print("Select an option (1, 2, 3, 4) ");
int choice = in.nextInt();
switch (choice)
{
   case 1:
      . . .
   case 2:
      . . .
   case 3:
      . . .
   case 4:
      . . .
   default:
      System.out.println("Bad input");
}
```





Switch case

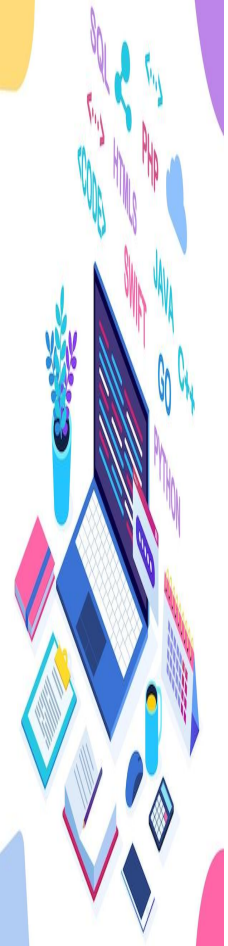
The break statement is optional. If you omit the break, execution will continue on into the next case. It is sometimes desirable to have multiple cases without break statements between them.

Example:

```
// In a switch, break statements are optional.
class MissingBreak {
  public static void main(String args[]) {
    for(int i=0; i<12; i++)
      switch(i) {
        case 0:
        case 1:
        case 2:
        case 3:
        case 4:
          System.out.println("i is less than 5");
          break;
        case 5:
        case 6:
        case 7:
        case 8:
        case 9:
          System.out.println("i is less than 10");
          break;
        default:
          System.out.println("i is 10 or more");
  } } }
```

Output:

```
i is less than 5
i is less than 5
i is less than 5
i is less than 5
i is less than 5
i is less than 10
i is less than 10
i is less than 10
i is less than 10
i is less than 10
i is 10 or more
i is 10 or more
```

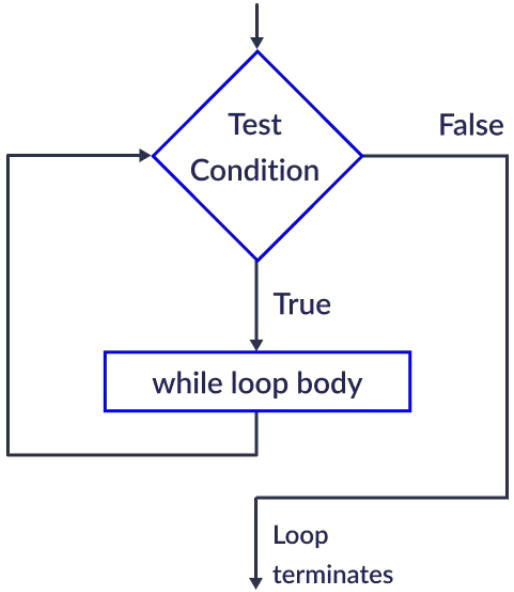


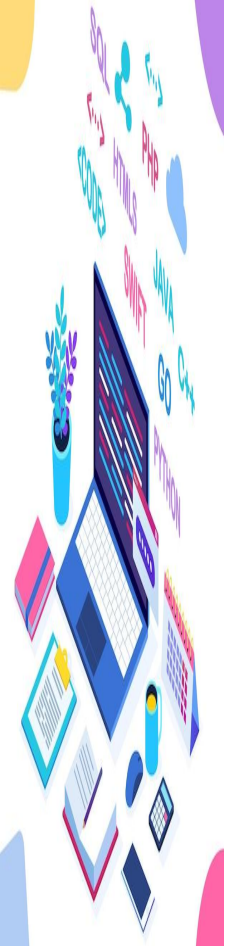
While loop

The while loop executes a statement (which may be a block statement) while a condition is true.

Syntax :

```
while(condition)
 {
        Body of the loop
 }
```

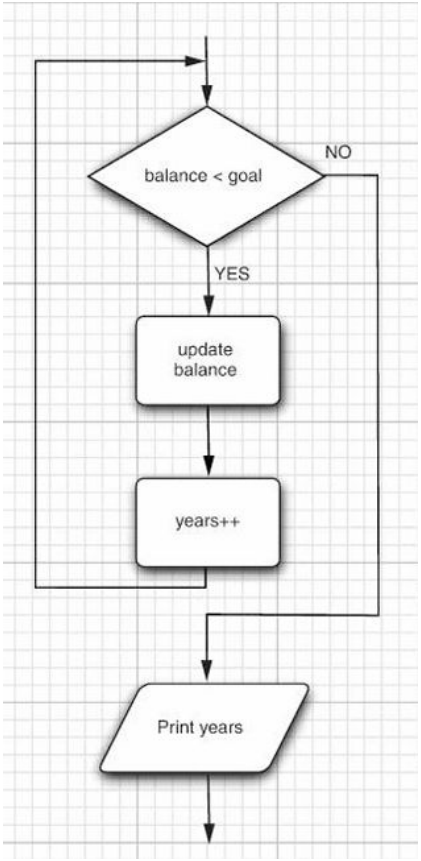


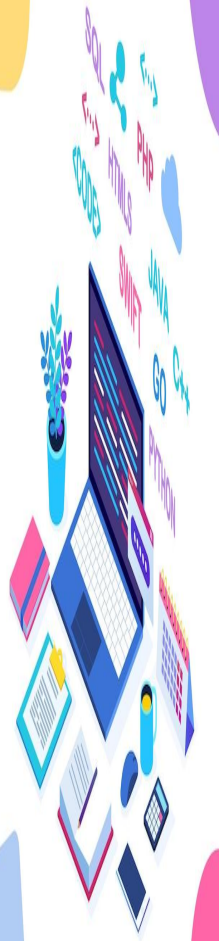


While loop

Example:

```
while (balance < goal)
{
   balance += payment;
   double interest = balance * interestRate / 100;
   balance += interest;
   years++;
}
System.out.println(years + " years.");
```



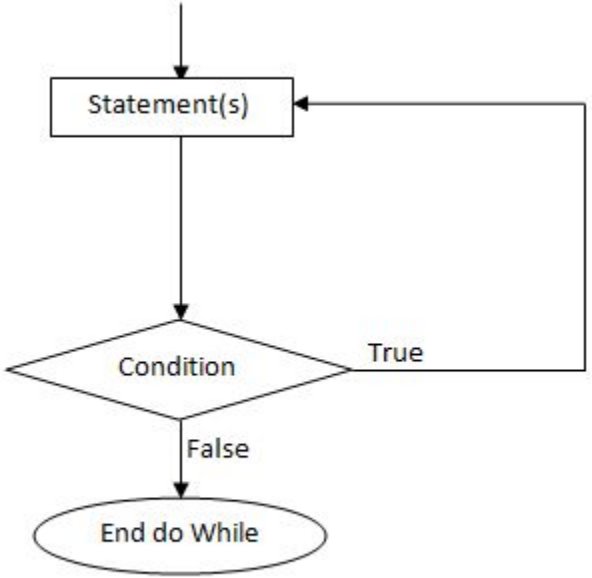


do-while loop

This loop executes the statement (which is typically a block) and only then tests the condition. If it's true, it repeats the statement and retests the condition, and so on.

Syntax :

```
do
 {
 Body of the loop
 } while(boolean expression);
```

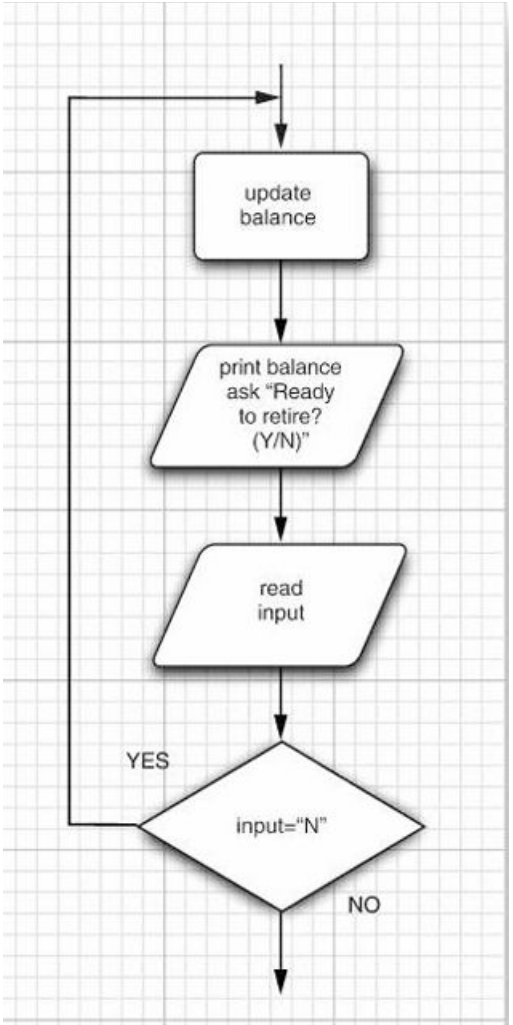


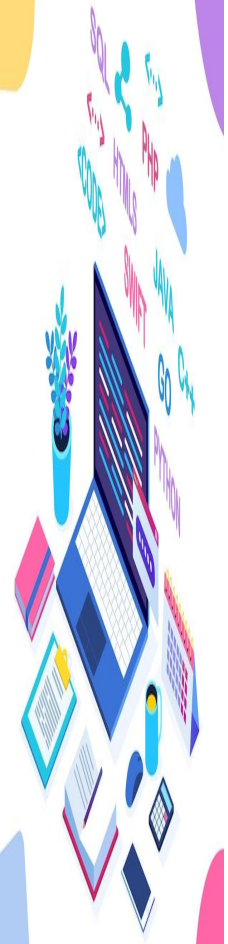


do-while loop

Example:

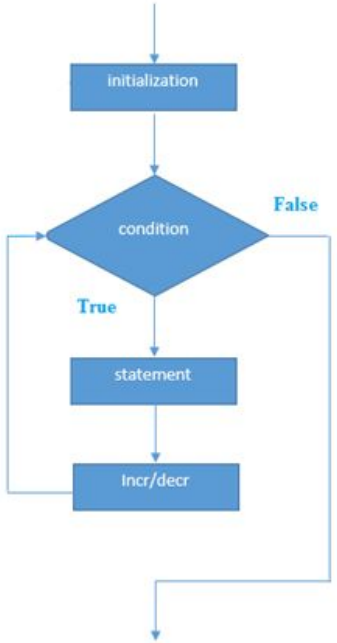
```
 do
{
   balance += payment;
   double interest = balance * interestRate / 100;
   balance += interest;
   years++;
   // print current balance
   . . .
   // ask if ready to retire and get input
   . . .
}
while (input.equals("N"));
```





For loop

- When the loop first starts, the initialization portion of the loop is executed. Generally, this is an expression that sets the value of the loop control variable, which acts as a counter that controls the loop.
- It is important to understand that the initialization expression is executed only once.
- Next, condition is evaluated. This must be a Boolean expression. It usually tests the loop control variable against a target value.
- If this expression is true, then the body of the loop is executed.
- Next, the iteration portion of the loop is executed. This is usually an expression that increments or decrements the loop control variable. The loop then iterates, first evaluating the conditional expression, then executing the body of the loop, and then executing the iteration expression with each pass. This process repeats until the controlling expression is false.



Syntax :

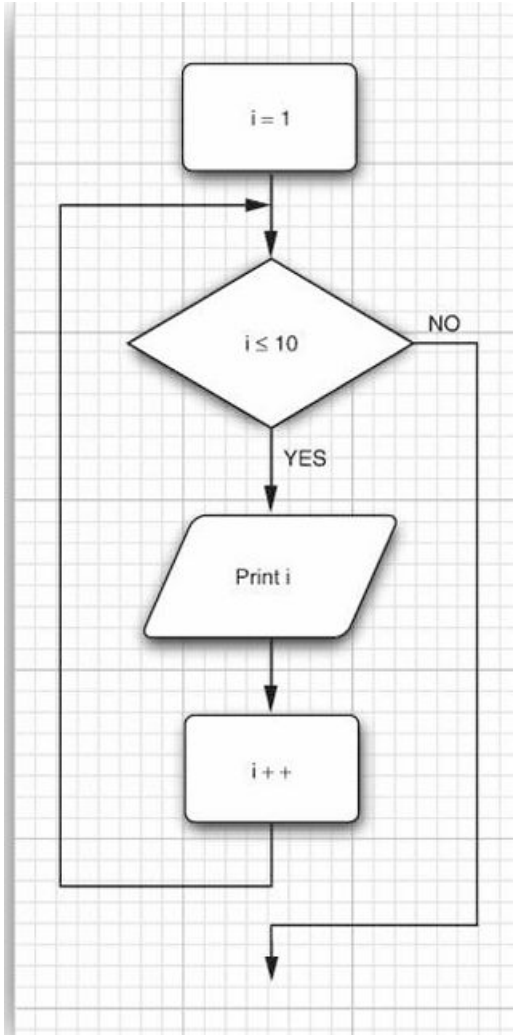
```
for(initialization;condition;increment/decrement)
{
Body of the loop
}
```

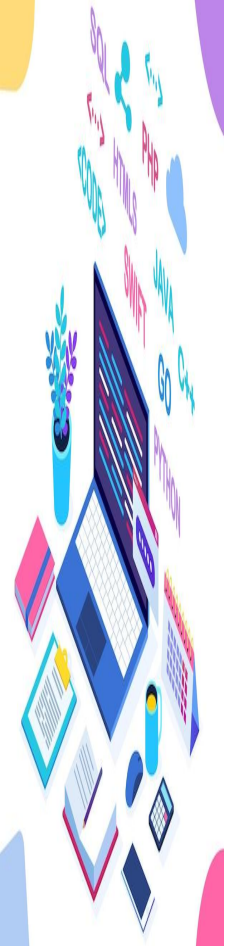


For loop

Example:

```
 for (int i = 1; i <= 10; i++)
{
   System.out.println(i);
}
```





Enhanced for loop

Syntax:

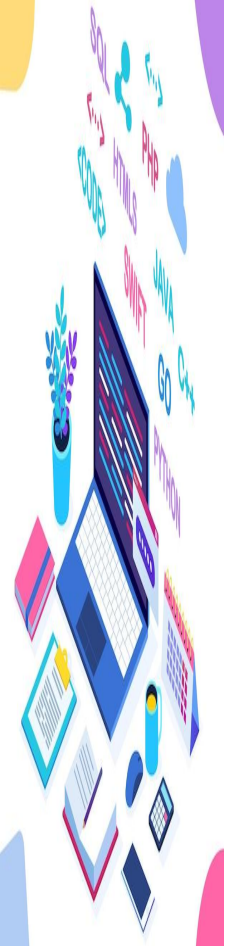
```
for(declaration : expression)
{
Body of loop
}
```

Example:

```
public class Sample {
public static void main(String[] args)
{
int [] numbers = {10, 20, 30, 40, 50};
for(int i : numbers )
{
System.out.println( "i: "+i );
}
}
}
```

Output:

i:10
i:20
i:30
i:40
i:50



Break statement

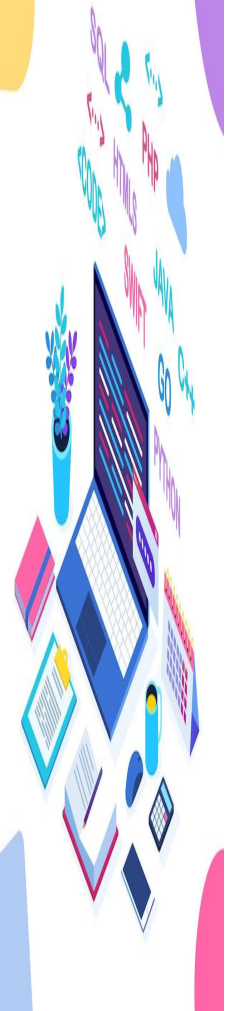
- While the execution of program, the break statement will terminate the iteration or switch case block
- When a break statement is encountered in a loop, the loop is exited and the program continues with the statements immediately following the loop
- When the loops are nested, the break will only terminate the corresponding loop body

Example:

```
public class Sample{
public static void main(String[] args)
{
for (int  i=1; i<=5; i++ )
{
if(i==2)
break;
System.out.println("i: "+i);
}
}
}
```

Output:

i:1



continue statement

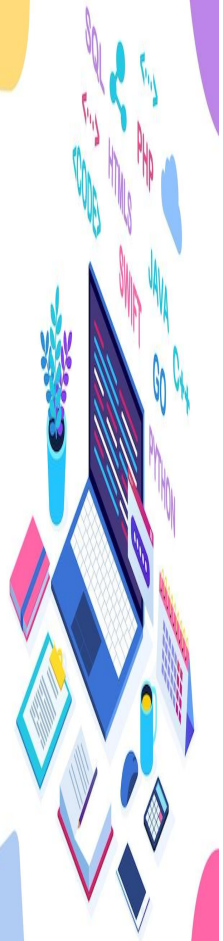
- The continue statement skips the current iteration of a loop
- In while and do loops, continue causes the control to go directly to the test-condition and then continue the iteration process
- In case of for loop, the increment section of the loop is executed before the test condition is evaluated

Example:

```
public class Sample {
public static void main(String[] args)
{
int [] numbers = {1, 2, 3, 4, 5};
for(int i : numbers )
{
if( i == 3 )
{
continue;
}
System.out.println( "i: "+i );
}
}
}
```

Output:

i: 1
i:2
i:4
i:5

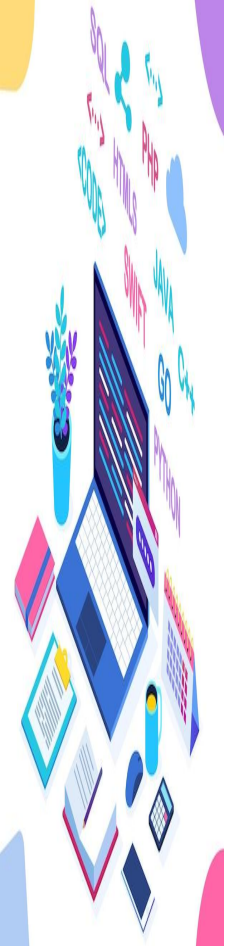


Arrays

- An array is a container object that holds a fixed number of values of a single type
- When an array is created, the length of an array is fixed
- Array elements are automatically initialized with the default value of their type, When an array is created
- Array can be created using the new keyword

Example:

  - int[] x = new int[5]; // defining an integer array for 5 blocks
- Alternatively, we can create and initialize array as below format
  - int[] x = {10, 20, 30};
  - int[] x = new int[]{10, 20, 30};
- Here the length of an array is determined by the number of values provided between {and}



Arrays

- The built-in length property determines the size of any array

Example:

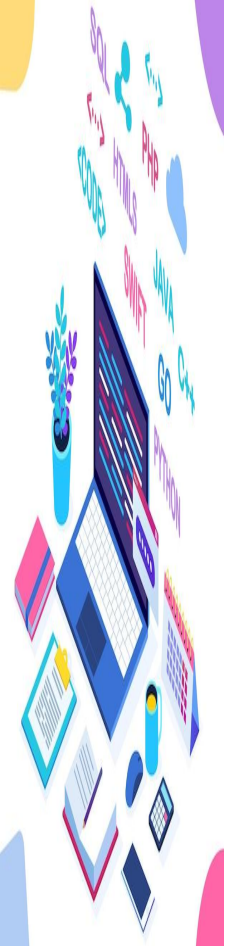
```
int[] x = new int[10];
int x_len = x.length;
```

Program:

```
public class ArrayDemo
{
public static void main(String[] args) {
        int[] x; // declares an array of integers
        x = new int[5]; // allocates memory for 5integers
        x[0] = 11;
        X[4] = 22;
        System.out.println("Element at index 0: " + x[0]);
        System.out.println("Element at index 1: " + x[1]);
        System.out.println("Element at index 4: " + x[4]);
        }
}
```

Output:

Element at index 0: 11
Element at index 1: 0
Element at index 4: 22

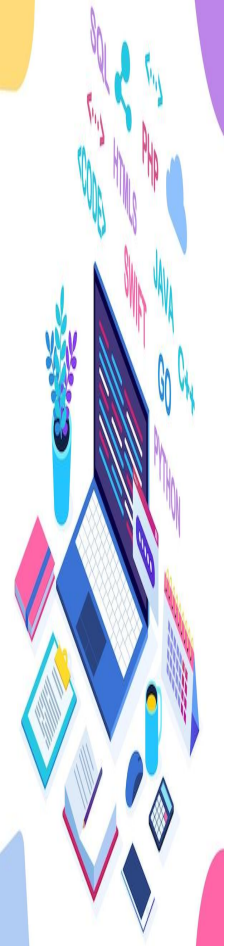


Array Bounds, Array Resizing

- Array subscripts begin with 0
- Can't access an array element beyond the range
- Can't resize an array.
- Can use the same reference variable to refer new array

```
int x[] = new int [5];
x= new int [10];
```

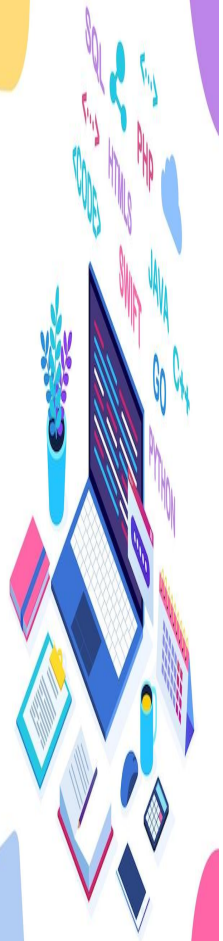
Array Bounds, Array Resizing



```
class Main {
    public static void main(String[] args) {
        int x[]=new int[5];
        x[0]=10;
        x[1]=20;
        x[2]=30;
        x[3]=40;
        x[4]=50;
        System.out.println("Before Resize");
        for(int i=0;i<x.length;i++)
        {
            System.out.println(x[i]);
        }
        x=new int[10];
        System.out.println("After Resize");
       for(int i=0;i<x.length;i++)
        {
            System.out.println(x[i]);
        }
    }
}
```

Output:

```
Before Resize
10
20
30
40
50
Before Resize
0
0
0
0
0
0
0
0
0
0
```



Two-Dimensional Arrays

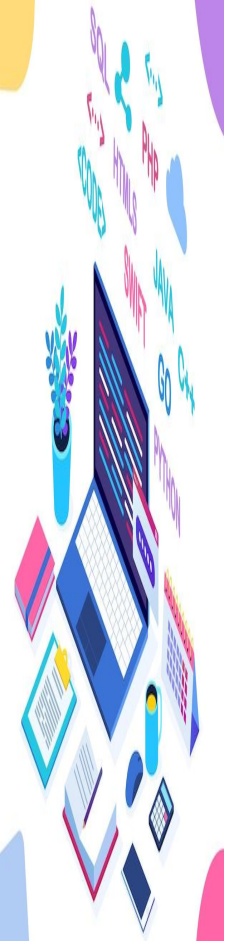
- Two-dimensional arrays are arrays of arrays
- Initializing two-dimensional arrays:

```
int[][] y = new int[3][3];
```

- The 1st dimension represent rows or number of one dimension, the 2nd dimension represent columns or number of elements in the each one dimensions
- The curly braces { } may also be used to initialize two dimensional arrays

Example:

```
int[][] y = { {1,2,3}, {4,5,6}, {7,8,9} };
int[][] y = new int[][] { {1,2,3}, {4,5,6}, {7,8,9} };
```



Two-Dimensional Arrays

- You can initialize the row dimension without initializing the columns but not vice versa

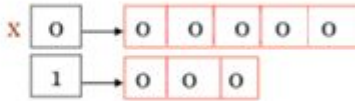
  int[][] x = new int[3][];

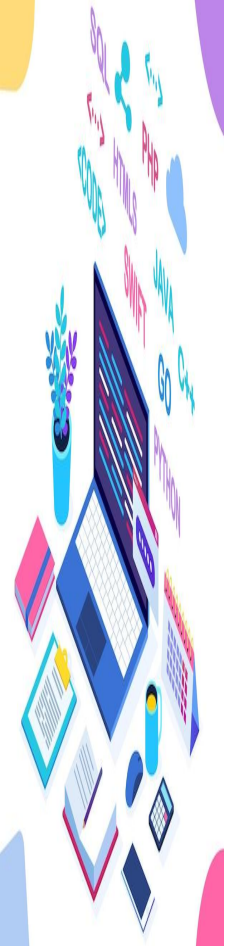
  int[][] x = new int[][3]; //error

- The length of the columns can vary for each row and initialize number of columns in each row

Ex1:

```
int [][]x = new int [2][];
x[0] = new int[5];
x[1] = new int [3];
```

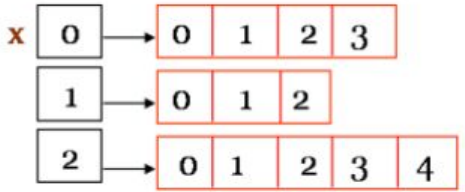


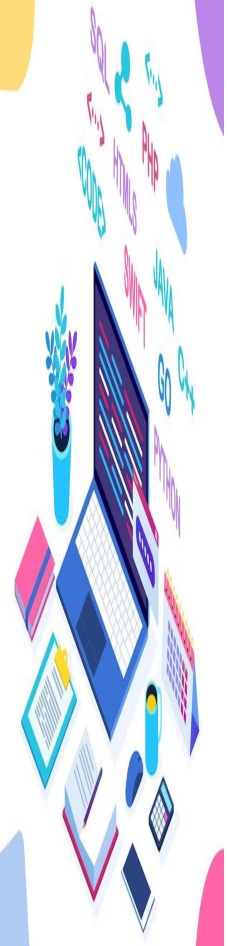


Two-Dimensional Arrays

Ex2:

```
int [][]x = new int
[3][];
x[0] = new
int[]{0,1,2,3};
x[1] = new int []{0,1,2};
x[2] = new
int[]{0,1,2,3,4};
```





Two-Dimensional Arrays

```
/* Program to under stand two-dimensional arrays */
class TwoDimDemo {
public static void main(String[] args) {
        int [][] x = new int[3][]; // initialize number of rows
        x[0] = new int[3]; // define number of columns in each row
        x[1] = new int[2];
        x[2] = new int[5];
        for(int i=0; i < x.length; i++) { // print array elements
                for (int j=0; j < x[i].length; j++) {
                        x[i][j] = i;
                        System.out.print(x[i][j]);
                }
                System.out.println();
        }
}
}
```

Output:

```
000
11
22222
```





RAJALAKSHMI ENGINEERING COLLEGE

WIPRO TALENTNEXT PROGRAM

FULL STACK DEVELOPMENT

MILE 1-SESSION 1

WIPRO TALENTNEXT PROGRAM



Mile 1

Core Java Programming



Mile 2

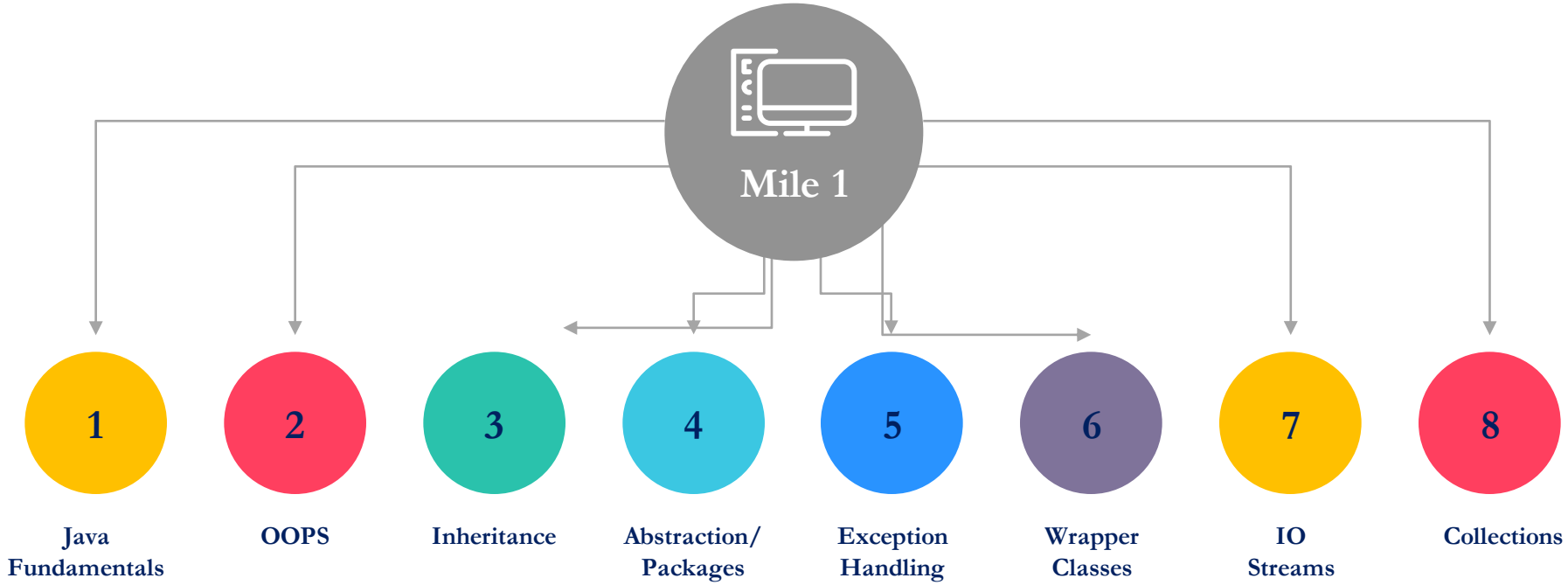
Threads and Database Connectivity



Mile 3

HTML,JSP,Servlets, AJAX,Springboot,etc

MILE 1-CORE JAVA PROGRAMMING



Java Fundamentals

01 Java Architecture

02 Language Basics

03 Control Statements

04 Arrays

Java Architecture-Quiz

What is the primary function of the Java Virtual Machine (JVM)?

A) To compile Java source code
B) To interpret bytecode into machine code
C) To handle memory management
D) To provide a graphical user interface

Question1

1
2
3

Java Architecture-Quiz

1

2

3

Which of the following is not a part of the Java Runtime Environment (JRE)?

Question 2

- A) JVM
- B) Java Class Libraries
- C) Java Compiler
- D) Class Loader

Java Architecture-Quiz

1

2

3

What is the purpose of the Just-In-Time (JIT) compiler in Java?

- A) To compile Java source code to bytecode
- B) To interpret bytecode at runtime
- C) To convert bytecode into native machine code at runtime
- D) To manage memory allocation and garbage collection

Question 3

Answers

1. To interpret bytecode into machine code
2. Java Compiler
3. To convert bytecode into native machine code at runtime

Language Basics-Quiz

Question1

Quiz Sample.java file contains class A, B and C. How many .class files will be created after compiling Sample.java?What is your observation?

Sample.java

```
class A { void m1() { } } class B { void m2() { } } class C { void m3() { }
```

1

2

3

Language Basics-Quiz

1

2

3

Question 2

What will be the result if you try to compile and execute the program ?

Sample.java

```
class Sample {
public static void main() {
System.out.println("Welcome");
}}
```

Answers

1. 3 class files
2. Runtime Error

Programs

Problem Statement:
In the city of Greenfield, it's observed that the population dynamics are influenced by the migration rate. Specifically, every 6 seconds, one person migrates into the city, and every 10 seconds, one person migrates out of the city. Given an initial population count, you need to estimate the population after a certain period.

```java
public class PopulationEstimator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Prompt the user to enter the initial population and the time period in seconds
        System.out.print("Enter the initial population: ");
        int initialPopulation = scanner.nextInt();
        System.out.print("Enter the time period in seconds: ");
        int timeSeconds = scanner.nextInt();

        // Calculate the number of people migrating in and out
        int peopleIn = timeSeconds / 6;
        int peopleOut = timeSeconds / 10;

        // Calculate the estimated population after the given time period
        int estimatedPopulation = initialPopulation + peopleIn - peopleOut;

        // Print the estimated population
        System.out.println(estimatedPopulation);
    }
}
```

Control Statements-Quiz

Question1

What are the three kinds of Control Statements?

1

2

3

Control Statements-Quiz

Question 2

Suppose you are writing code for a for loop that must execute three times. Which is the correct declaration for the loop?

1

2

3

```
for (int i < 4; i = 1; i++)
for (int i = 0; i < 4; i++)
for (int i = 1; i++; i < 4)
for (int i = 3; i >=1; i--)
```

Answers

1. Selection, Iteration and Jumping
2. for (int i = 3; i >=1; i--)

MCQS

Given the following code:

```
public class TestForSwitch {
 public static void main(String[] args) {
  for (int i = 0; i < 3; i++) {
   switch (i) {
   default:
    System.out.print("D");
   case 0:
    System.out.print("0");
   case 1:
    System.out.print("1");
   } } }}
```

What is the result of attempting to compile and run the program?

1. Prints: DDD
2. Prints: 01D
3. Prints: 01D01
4. Prints: 011D01

Prints: 011D01

MCQS

```
public class TestIf {
    public static void main(String[] args) {
        boolean bFlag = true;
        if (bFlag = false) {
            System.out.print("X");
        } else if (bFlag) {
            System.out.print("Y");
        } else {
            System.out.print("Z");
        }
    }
}
```

Prints: X
Prints: Y
Prints: Z
Compile-time error

Prints: Z

MCQS

```
int j = 1;
while (j < 10) {
    System.out.println(j + " ");
    j = j + j % 3;
}
```

1. 1 4 7
2. 1 4 7 10
3. 1 2 5 8
4. 1 2 4 5 7 8

1 2 4 5 7 8

MCQS

What will be the output of the following program?

```
class Main {
    public static void main(String args[]) {
            int x = 2;
            int y = 0;
            for ( ; y < 10; ++y) {
                    if (y % x == 0)
                            continue;
                    else if (y == 8)
                            break;
                    else
                            System.out.print(y + " ");
                    }
    }
}
```

1. 1 3 5 7 9
2. 1 3 5 7
3. Compile Time Error
4. Run Time Error

1 3 5 7 9

MCQS

What will be the output of the following program?

```
public class Main {
    public static void main(String args[])  {
       int i = 0;
        for (System.out.print("Wise"); i < 1; i++)
            System.out.print("Stark");
    }
}
```

1. StarkWise
2. Wise
3. Stark
4. WiseStark

WiseStark

MCQS

What will be the output of the following program?

```
public class Main {
    public static void main(String[] args) {
        boolean name = true;
        switch (name) {
            case true:
                System.out.println("Hello ");
            default:
                System.out.println("hi");
        }
    }
}
```

1. hi
2. Hello
3. Compile Time Error
4. Hello hi

Compile Time Error

MCQS

What will be the output of the following program?

```
public class Main {
        public static void main(String[] args) {
                for (char ch : "12345".toCharArray()) {
                        if (ch == '3') {
                                continue;
                        }
                        System.out.print(ch + " ");
                }
        }
}
```

1. 1 2 4 5
2. 1 2 3 4 5
3. 1 2 3 4
4. 1 2 3 5

1 2 4 5

MCQS

What will be the output of the following program?

```
public class Main {
    public static void main(String[] args) {
        for(int i = 0; i>5; ){
            System.out.println("Hello");
        }
    }
}
```

1. 5
2. 3
3. 4
4. No Output

No Output

Problem Statement 1

Samantha is a diligent math student who is exploring the world of programming. She is learning Java and has recently studied conditional statements. One day, her teacher gives her an interesting problem to solve, that takes a number as input and checks whether it is a multiple of 5 or 7.

Help her to complete the task.

Input Format :

The input consists of a single integer **N,** representing the number to be checked.

Output Format :

If the number is a multiple of 5 but not 7, the output prints "N is a multiple of 5".

If the number is a multiple of 7, the output prints "N is a multiple of 7".

Otherwise the output prints "N is neither multiple of 5 nor 7" where N is an entered integer.

Sample testcases :

Testcase 1 Input

10

Testcase 1 Output

10 is a multiple of 5

Testcase 2 Input

21

Testcase 2 Output

21 is a multiple of 7

Testcase 3 Input

37

Testcase 3 Output

37 is neither multiple of 5 nor 7

Solution

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int number = scanner.nextInt();
        if (number % 5 == 0) {
            System.out.println(number + " is a multiple of 5");
        } else if(number % 7 == 0){
            System.out.println(number + " is a multiple of 7");
        }
        else{
            System.out.println(number + " is neither multiple of 5 nor 7");
        }
        scanner.close();
    }
}
```

Problem Statement 2

Subha is tasked with developing a program to analyze and categorize a large dataset of numerical data points. As a part of her program, she needs to identify and extract all the even numbers from the dataset, which ranges from 2 to a specified integer(inclusive).
Help Subha to write this program using a 'for' loop.

Input Format :
The input consists of an integer N, representing the upper limit of the range.
Output Format :
The output displays the even numbers in the dataset separated by space, till the specified integer.

Sample testcases :

Testcase 1 Input
17
Testcase 1 Output
2 4 6 8 10 12 14 16

Testcase 2 Input
6
Testcase 2 Output
2 4 6

Testcase 3 Input
18
Testcase 3 Output
2 4 6 8 10 12 14 16 18

Solution

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        for (int i = 1; i <= n; i++) {
            if (i % 2 == 0) {
                System.out.print(i + " ");
            }
        }
    }
}
```

Problem Statement 3

Akash is tasked with developing a program that calculates and categorizes blood pressure based on the given systolic and diastolic readings.
The program should use the following classifications:

Low Blood Pressure: Systolic < 90 mm Hg or Diastolic < 60 mm Hg
Normal Blood Pressure: Systolic ≤ 120 mm Hg and Diastolic ≤ 80 mm Hg
Prehypertension: Systolic ≤ 140 mm Hg and Diastolic ≤ 90 mm Hg
Stage 1 Hypertension: Systolic ≤ 160 mm Hg and Diastolic ≤ 100 mm Hg
Stage 2 Hypertension: Otherwise

Write a program to assist Akash in computing and classifying blood pressure levels based on input readings. The program should use a 'switch-case' to deliver a tailored blood pressure category.

Input Format :
The input consists of two space-separated integers, representing the systolic blood pressure value **S** and diastolic blood pressure value **D,** respectively.
Output Format :
The output displays "Blood Pressure Category: " followed by the blood pressure category based on the provided input.

Sample testcases :

Testcase 1 Input
50 85
Testcase 1 Output
Blood Pressure Category: Low Blood Pressure

Testcase 2 Input
112 70
Testcase 2 Output
Blood Pressure Category: Normal Blood Pressure

Testcase 3 Input
135 86
Testcase 3 Output
Blood Pressure Category: Prehypertension

Solution

```
import java.util.Scanner;
class BloodPressureCalculator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int systolic = scanner.nextInt();
        int diastolic = scanner.nextInt();
        int pulsePressure = systolic - diastolic;
        int category;
        if (systolic < 90 || diastolic < 60) {
            category = 1;
        } else if (systolic <= 120 && diastolic <= 80) {
            category = 2;
        } else if (systolic <= 140 && diastolic <= 90) {
            category = 3;
        } else if (systolic <= 160 && diastolic <= 100) {
            category = 4;
        } else {
            category = 5;
        }
```

```
System.out.print("Blood Pressure Category: ");
        switch (category) {
            case 1:
                System.out.println("Low Blood Pressure");
                break;
            case 2:
                System.out.println("Normal Blood Pressure");
                break;
            case 3:
                System.out.println("Prehypertension");
                break;
            case 4:
                System.out.println("Stage 1 Hypertension");
                break;
            case 5:
                System.out.println("Stage 2 Hypertension");
                break;
          }
        scanner.close();
    }
}
```

Problem Statement 4

Alice, a math teacher, is creating a programming exercise to help her students practice multiplying all the odd digits of a given integer. She wants to create a simple program that takes a positive integer input and finds the product of its odd digits.

Help Alice to complete the program using the 'while' loop.

Input Format :
The input consists of a single positive integer **N.**
Output Format :
The output displays the product of odd digits in the given integer **N.**
If no odd digits are found in **N,** display "No odd digits found".

Sample testcases :

Testcase 1 Input
732
Testcase 1 Output
21

Testcase 2 Input
2468
Testcase 2 Output
No odd digits found

Testcase 3 Input
467
Testcase 3 Output
7

Solution

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int n = scanner.nextInt();

        int product = 1;
        boolean oddDigitFound = false;

        while (n > 0) {
            int digit = n % 10;
            if (digit % 2 != 0) {
                product *= digit;
                oddDigitFound = true;
            }
            n /= 10;
        }
```

```
        if (oddDigitFound) {
            System.out.println(product);
        } else {
            System.out.println("No odd digits found");
        }

        scanner.close();
    }
}
```

Problem Statement 5

You are a software developer at a security firm and have been tasked with creating a system to generate a secure PIN for user authentication. The PIN should be generated using three variables: alpha, beta, and gamma. Each of these variables holds a positive integer value provided by the user. The system should use these three numbers to compute a secure PIN.

The rules for generating the PIN are as follows:
1.Calculate the sum of alpha and beta.
2.Multiply the result by gamma.
3.Add the digit sum of alpha to the product obtained in step 2.
4.The PIN should be a 6-digit number. If the resulting number has more than 6 digits, truncate it to the first 6 digits. If it has fewer than 6 digits, pad it with leading zeros to make it exactly 6 digits long.

Write a Java program that prompts the user to input the values for alpha, beta, and gamma, and then generates and prints the secure PIN based on the rules above.

Solution

```java
import java.util.Scanner;

public class PinGenerator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Prompt the user to enter the values for alpha, beta, and gamma
        System.out.print("Enter the value for alpha: ");
        int alpha = scanner.nextInt();
        System.out.print("Enter the value for beta: ");
        int beta = scanner.nextInt();
        System.out.print("Enter the value for gamma: ");
        int gamma = scanner.nextInt();

        // Calculate the sum of alpha and beta
        int sumAlphaBeta = alpha + beta;

        // Multiply the result by gamma
        int product = sumAlphaBeta * gamma;

        // Add the digit sum of alpha to the product
        int digitSumAlpha = digitSum(alpha);
        int pinValue = product + digitSumAlpha;

        // Ensure the PIN is exactly 6 digits long
        int pin = pinValue % 1000000;

        // Print the generated PIN with leading zeros if necessary
        System.out.printf("Generated PIN: %06d\n", pin);
    }

    // Method to calculate the digit sum of a number
    public static int digitSum(int number) {
        int sum = 0;
        while (number > 0) {
            sum += number % 10;
            number /= 10;
        }
        return sum;
    }
}
```

Problem Statement 6

Louise and Richard have developed a numbers game. They pick a number and check to see if it is a power of 2. If it is, they divide it by 2. If not, they reduce it by the next lower number which is a power of 2. Whoever reduces the number to 1 wins the game. Louise always starts.

Given an initial value, determine who wins the game.

Example: n=132

It's Louise's turn first. She determines that 132 is not a power of 2. The next lower power of 2 is 128, so she subtracts that from 132 and passes 4 to Richard. 4 is a power of 2, so Richard divides it by 2 and passes 2 to Louise. Likewise, 2 is a power so she divides it by 2 and reaches 1. She wins the game.

Update If they initially set counter to 1, Richard wins. Louise cannot make a move so she loses.

Function Description

Complete the *counterGame* function in the editor below.

counterGame has the following parameter(s):

- *int n:* the initial game counter value

Returns

- *string:* either Richard or Louise

Input Format

The first line contains an integer , the number of testcases.

Each of the next lines contains an integer , the initial value for each game

Constraints

- $1 <= t <= 10$
- $1 <= n <= n^{64} - 1$

Sample Input 0

1

6

Sample Output 0

Richard

Explanation 0

- 6 is not a power of 2 so Louise reduces it by the largest power of 2 less than 6:

  6-4=2.

- 2 is a power of 2 so Richard divides by 2 to get 1 and wins the game.

Solution

```
import java.util.Scanner;
 public class CounterGame {
       public static String counterGame(long n) {
       int moves = 0;
       while (n > 1) {
          if ((n & (n - 1)) == 0) {
             n /= 2;
          } else {
             n -= Long.highestOneBit(n);
          }
          moves++;
       }
        return (moves % 2 == 0) ? "Richard" : "Louise";
    }
```

```
 public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int t = scanner.nextInt();
         for (int i = 0; i < t; i++) {
            long n = scanner.nextLong();
            System.out.println(counterGame(n));
        }
         scanner.close();
    }
}
```

Thank you

Arrays

- An array is a container object that holds a fixed number of values of a single type
- When an array is created, the length of an array is fixed
- Array elements are automatically initialized with the default value of their type, When an array is created
- Array can be created using the new keyword

- Ex
  - int[] x = new int[5];
    - // defining an integer array for 5 blocks
- Alternatively, we can create and initialize array as below format
  - int[] x = {10, 20, 30};
  - int[] x = new int[]{10, 20, 30};
- Here the length of an array is determined by the number of values provided between {and}

- The built-in length property determines the size of any array
- Ex:
  - int[] x = new int[10];
  - int x_len = x.length;

```
public class ArrayDemo
{
public static void main(String[] args) {
        int[] x; // declares an array of integers
        x = new int[5]; // allocates memory for 5integers
        x[0] = 11;
        X[4] = 22;
        System.out.println("Element at index 0: " + x[0]);
        System.out.println("Element at index 1: " + x[1]);
        System.out.println("Element at index 4: " + x[4]);
        }
}
```

Output:

```
Element at index 0: 11
Element at index 1: 0
Element at index 4: 22
```

Array Bounds, Array Resizing

- Array subscripts begin with 0
- Can't access an array element beyond the range
- Can't resize an array.
- Can use the same reference variable to refer new array
  - int x[] = new int [5];

Array Resize Example

```
class Main {
   public static void main(String[] args) {
      int x[]=new int[5];
      x[0]=10;
      x[1]=20;
      x[2]=30;
      x[3]=40;
      x[4]=50;
      System.out.println("Before Resize");
      for(int i=0;i<x.length;i++)
      {
         System.out.println(x[i]);
      }
      x=new int[10];
      System.out.println("After Resize");
     for(int i=0;i<x.length;i++)
      {
         System.out.println(x[i]);
      }
   }
}
```

Two-Dimensional Arrays

- Two-dimensional arrays are arrays of arrays
- Initializing two-dimensional arrays:
  - int[][] y = new int[3][3];
- The 1st dimension represent rows or number of one dimension, the 2nd dimension represent columns or number of elements in the each one dimensions

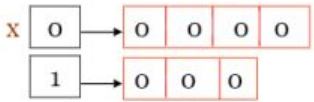
- The curly braces { } may also be used to initialize two dimensional arrays
- Ex:
  - int[][] y = { {1,2,3}, {4,5,6}, {7,8,9} };
  - int[][] y = new int[][] { {1,2,3}, {4,5,6}, {7,8,9} };

Two-Dimensional Arrays (Contd.).

- You can initialize the row dimension without initializing the columns but not vice versa
  - int[][] x = new int[3][];
  - int[][] x = new int[][3]; //error
- The length of the columns can vary for each row and initialize number of columns in each row

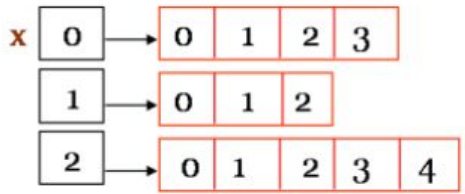
Ex1:

```
int [][]x = new int [2][];
x[0] = new int[5];
x[1] = new int [3];
```



Ex2:

```
int [][]x = new int
[3][];
x[0] = new
int[]{0,1,2,3};
x[1] = new int []{0,1,2};
x[2] = new
int[]{0,1,2,3,4};
```



```
/* Program to under stand two-dimensional arrays */
class TwoDimDemo {
public static void main(String[] args) {
        int [][] x = new int[3][]; // initialize number of rows
        x[0] = new int[3]; // define number of columns in each row
        x[1] = new int[2];
        x[2] = new int[5];
        for(int i=0; i < x.length; i++) { // print array elements
                for (int j=0; j < x[i].length; j++) {
                        x[i][j] = i;
                        System.out.print(x[i][j]);
                }
                System.out.println();
        }
}
}
```

Output:

```
000
11
22222
```

Arrays-Quiz

Which of the following is a valid way to initialize an array in Java?

Question1

- A) `int[] a = new int[3]{1, 2, 3};`
- B) `int[] a = {1, 2, 3};`
- C) `int a[] = new int[3]; a[0] = 1; a[1] = 2; a[2] = 3;`
- D) Both B and C

1

2

3

Arrays-Quiz

```
class Sample {
    public static void main(String[] args) {
        int[] a = new int[5];
        for (int i = 0; i < 5; i++) {
            System.out.println(a[i]);
        }    }}
```

Question 2

1

2

3

Problem Statement 1

You are developing a warehouse management system for a shipping company. The system uses an integer array to represent the weights of packages in a specific order. To verify that the weight capacity is not exceeded, the program needs to calculate the sum of the weights of the first and last packages in the list.

Task: Write a code to calculate the sum of the weights of the first and last packages in the list. The program should take an integer array as input and return the total weight of the first and last packages.

Input Format :

The first line of the input is an integer **N** representing the size of the array.

The second line of the input is **N** space-separated integer values.

Output Format :

The output is displayed in the following format:

Testcase 1 Input

5

10 20 30 40 50

Testcase 1 Output

Sum of the first and last elements: 60

Solution

```
import java.util.*;
class Sample
{
    public static void main(String a[])
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int arr[]= new int[n];
        for (int i=0; i<n; i++){
            arr[i]= sc.nextInt();
        }
        System.out.println("Sum of the first and last elements: "+(arr[0]+arr[n-1]));
    }
}
```

Problem Statement 2

Aasha wants to write a program to print the four corner elements of a 2D array. These corners represent the elements at the extremities of the array.

Write a program that takes user input, processes the data and prints these corner elements.

Input Format :

The first line of input consists of an integer **N**, representing the number of rows.

The second line consists of an integer **M**, representing the number of columns.

The following lines **N** lines consist of **M** space-separated integers, representing the elements of the 2D array.

Output Format :

The output prints the corner elements of the 2D array, in the following format:

"Top-left: " followed by the top left corner element.

"Top-right: " followed by the top right corner element.

"Bottom-left: " followed by the bottom left corner element.

"Bottom-right: " followed by the bottom right corner element.

If the array does not have four corners (e.g., due to small dimensions), print "The array does not have four corners."

Testcase 1 Input

3

4

1 2 3 4

5 6 7 8

9 1 4 5

Testcase 1 Output

Top-left: 1

Top-right: 4

Bottom-left: 9

Bottom-right: 5

Solution

```
import java.util.*;
class Sample
{
    public static void main(String a[])
    {
        Scanner sc = new Scanner(System.in);
        int r=sc.nextInt();
        int c=sc.nextInt();
        int m[][]=new int[r][c];
        for (int i=0; i<r; i++)
        {
            for(int j=0; j<c; j++)
            {
                m[i][j]=sc.nextInt();
            }
        }
```

```
if(r==1 || c==1)
            System.out.println("The array does not have four
corners.");
        else
            {
                System.out.println("Top-left: "+m[0][0]);
                System.out.println("Top-right:"+m[0][c-1]);
                System.out.println("Bottom-left:"+m[r-1][0]);
                System.out.println("Bottom-right:"+m[r-1][c-1]);
            }
    }
}
```

Problem Statement 3

In a virtual treasure hunt game, adventurers embark on a journey to find hidden gems. Along their path, they discover a series of mystical stones, each represented by an integer value. These stones are collected in an array. However, the adventurers can only pick stones at odd-numbered positions.

Write a program to get the array from the user and display only the odd indices present in the array.

Input Format :

The first line of input consists of an integer **N,** representing the size of the original array.

The second line consists of **N** space-separated integers, representing the elements of the array.

Output Format :

The output prints the elements at odd indices (index position starts from 1).

Sample testcases :

Testcase 1 Input

5

10 20 30 40 50

Testcase 1 Output

10 30 50

Solution

```
import java.util.Scanner;

public class TreasureHunt {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Input: Size of the array
        int N = sc.nextInt();
        int[] stones = new int[N];

        // Input: Array elements
        for (int i = 0; i < N; i++) {
            stones[i] = sc.nextInt();
        }
        // Output: Elements at odd-numbered positions (1-based odd index = 0-based even index)
        for (int i = 0; i < N; i += 2) {
            System.out.print(stones[i] + " ");
        }
        sc.close();
    }
}
```

Problem Statement 4

Shaun is working on a project to analyze sales data for various products in a retail store. Each element in a 2D matrix represents the sales figures for different products across various regions. To focus her analysis, she needs to delete the row and column containing a specific product's sales figure.

Write a program to help Shaun delete the row and column containing a specific product's sales figure.

Input Format :

The first line of the input consists of two space-separated integers, **R** and **C**, representing the number of rows and columns in the matrix, respectively.

The following **R** lines each contain, **C** space-separated integers representing the elements of the matrix (sales figures).

The last line consists of an integer **N**, representing the specific product's sales figure to be deleted.

Output Format :

The output prints the modified matrix after deleting the row and column containing the specified element (product's sales figure).

Testcase 1 Input

3 3

1 2 3

4 5 6

7 8 9

2

Testcase 1 Output

4 6

7 9

Testcase 2 Input

2 2

1 10

12 2

Testcase 2 Output

1

Solution

```
class Main
{
    public static void main(String[] args)
    {
        Scanner scanner = new Scanner(System.in);
        int rows = scanner.nextInt();
        int cols = scanner.nextInt();
        int r=0,c=0;
        int[][] a = new int[rows][cols];
        for (int i = 0; i < rows; i++){
            for (int j = 0; j < cols; j++)
            {
                a[i][j] = scanner.nextInt();
            }
        }
    int element = scanner.nextInt();
         for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++){
              if(a[i][j]==element)
              {
                 r=i;
                 c=j;
```

```
        }
    }
}


for (int i = 0; i < rows; i++)
        {
            for (int j = 0; j < cols; j++)
            {
                if(i==r || j==c)
                {
                    continue;
                }
                System.out.println(a[i][j]);
            }
        }
    }
}
```

Problem Statement 6

Watson gives Sherlock an array of integers. His challenge is to find an element of the array such that the sum of all elements to the left is equal to the sum of all elements to the right. For instance, given the array arr = [5, 6, 8, 11], 8 is between two subarrays that sum to 11. If your starting array is [1], that element satisfies the rule as left and right sum to 0.

You will be given arrays of integers and must determine whether there is an element that meets the criterion. It should return a string, either YES if there is an element meeting the criterion or NO otherwise.

It has the following:

- arr: an array of integers

Input Format

The first line contains T, the number of test cases.

The next T pairs of lines each represent a test case.

- The first line contains n, the number of elements in the array arr.
- The second line contains n space-separated integers arr[i] where $0 \leq i < n$.

Solution

```
import java.util.*;
public class PalindromeDigitChecker {
    public static void main(String[] args) {
        System.out.println("Enter the integer: ");
        // Create Scanner object
        Scanner s = new Scanner(System.in);
        int n;
        // Read the next integer from the screen
        n= s.nextInt();
        int a[]=new int[n];
        System.out.println("Enter elements");
        for(int i=0;i<n;i++)
        {
            a[i]=s.nextInt();
        }
    int me=n/2;
    int ls=0,rs=0;
```

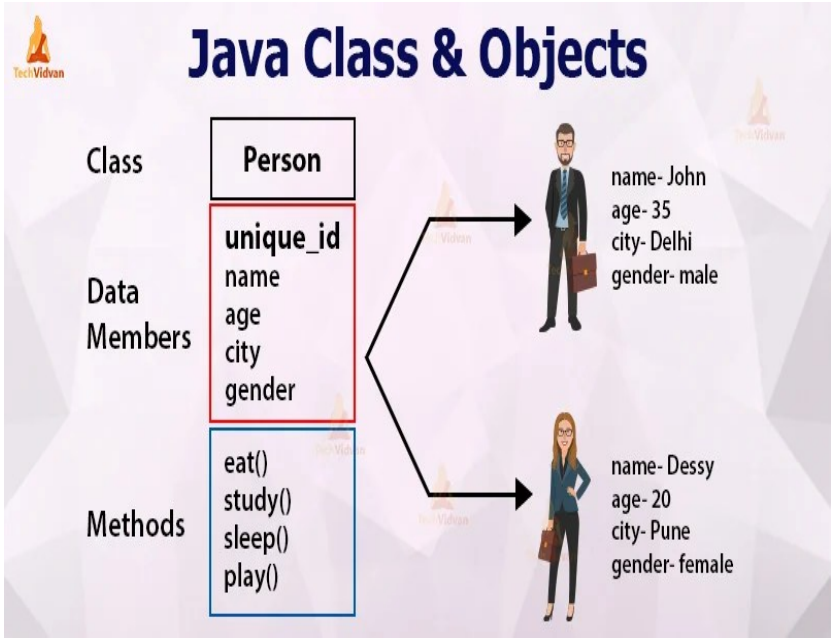
```
for(int i=0;i<n;i++)
    {
        if(i<me)
        {
            ls=ls+a[i];
        }
        else if(i>me)
        {
            rs=rs+a[i];
        }

    }
    System.out.println(ls +" " + rs);
    if(ls==rs)
    {
        System.out.println("YES");
    }
    else
    {
        System.out.println("No");
    }
    }
}
```

Classes and Objects

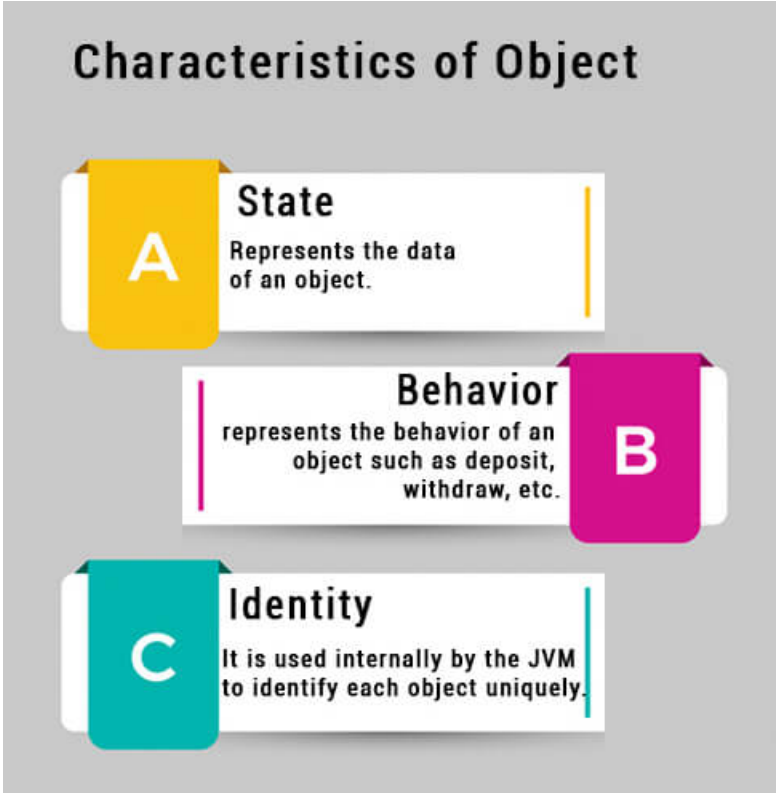
Class

- A class is a group of objects which have common properties.
- It is a template or blueprint from which objects are created.
- It is a logical entity.



Object

- A Java object is a member (also called an instance) of a Java class.
- Each object has an identity, a behavior and a state. The state of an object is stored in fields (variables), while methods (functions) display the object's behavior.
- Objects are created at runtime from templates, which are also known as classes.


Characteristics of Object
A
State
Represents the data of an object.
Behavior
represents the behavior of an object such as deposit, withdraw, etc.
B
C
Identity
It is used internally by the JVM to identify each object uniquely.

GENERAL FORM

```
class classname
{
 type instance-variable1;
// ...
 type instance-variableN;
 type methodname1(parameter-list)  {
        // body of method
 }
 type methodname2(parameter-list)  {
        // body of method
 }
 // ...
 type methodnameN(parameter-list)  {
        // body of method
 }
}
```

A Simple Class

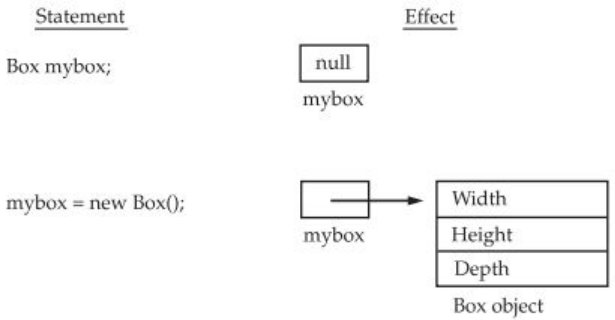
```
class Box {
    double width;
    double height;
    double depth;
}
```

Declaring Objects

- Box mybox = new Box();

Or

- Box mybox; // declare reference to object
- mybox = new Box(); // allocate a Box object



```
// This class declares an object of type Box.
class BoxDemo
{
 public static void main(String args[])
 {
        Box mybox = new Box();
        double vol;

        // assign values to mybox's instance variables
        mybox.width = 10;
        mybox.height = 20;
        mybox.depth = 15;

        // compute volume of box
        vol = mybox.width * mybox.height * mybox.depth;

        System.out.println("Volume is " + vol);
 }
}
```

```
// This program declares two Box objects.

class Box {
  double width;
  double height;
  double depth;
}

class BoxDemo2 {
  public static void main(String args[]) {
    Box mybox1 = new Box();
    Box mybox2 = new Box();
    double vol;

    // assign values to mybox1's instance variables
    mybox1.width = 10;
    mybox1.height = 20;
    mybox1.depth = 15;

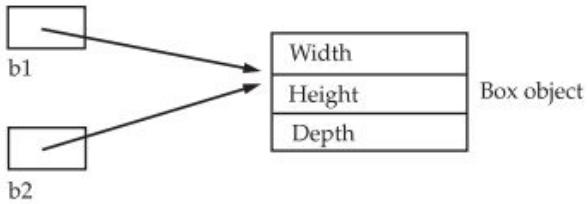
    /* assign different values to mybox2's
       instance variables */
    mybox2.width = 3;
    mybox2.height = 6;
    mybox2.depth = 9;

    // compute volume of first box
    vol = mybox1.width * mybox1.height * mybox1.depth;
    System.out.println("Volume is " + vol);

    // compute volume of second box
    vol = mybox2.width * mybox2.height * mybox2.depth;
    System.out.println("Volume is " + vol);
  }
}
```

Assigning Object Reference Variables

- Box b1 = new Box();
- Box b2 = b1;



Introducing Methods

- Methods is a large one because Java gives them so much power and flexibility.

- This is the general form of a method:

```
type name(parameter-list) {
        // body of method
}
```

Adding a Method to the Box Class

```
class Box {
        double width;
        double height;
        double depth;
        void volume()
        {
        System.out.println(“Welcome to volume method”);
        }
}
```

```
class BoxDemo {
 public static void main(String args[])  {
      Box mybox = new Box();
      double vol;

      // assign values to mybox's instance variables
      mybox.width = 10;
      mybox.height = 20;
      mybox.depth = 15;

      // call volume method
      mybox.volume();
 }
}
```

```
// This program uses a parameterized method.

class Box {
  double width;
  double height;
  double depth;

  // compute and return volume
  double volume() {
    return width * height * depth;
  }

  // sets dimensions of box
  void setDim(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
  }
}
```

```
public static void main(String args[]) {
  Box mybox1 = new Box();
  Box mybox2 = new Box();
  double vol;

  // initialize each box
  mybox1.setDim(10, 20, 15);
  mybox2.setDim(3, 6, 9);

  // get volume of first box
  vol = mybox1.volume();
  System.out.println("Volume is " + vol);

  // get volume of second box
  vol = mybox2.volume();
  System.out.println("Volume is " + vol);
}
```

Constructors

- While designing a class, the class designer can define within the class, a special method called ‘constructor’
- Constructor is automatically invoked whenever an object of the class is created
- Rules to define a constructor

✔ A constructor has the same name as the class name

✔ A constructor should not have a return type

✔ A constructor can be defined with any access specifier (like private, public)

✔ A class can contain more than one constructor, So it can be overloaded

```
class Sample
{
	private int id;
	Sample()
	{
		id = 101;
		System.out.println("Default constructor, with ID: "+id);
	}
	Sample(int no)
	{
		id = no;
		System.out.println("One argument constructor,with ID: "+ id);
	}
}
```

```
public class ConstDemo
{
    public static void main(String[] args)
    {
        Sample s1 = new Sample();
        Sample s2 = new Sample(102);
    }
}
```

Output:
Default constructor, with ID: 101
One argument constructor,with ID: 102

Overloading Constructors

- In addition to overloading normal methods, you can also overload constructor methods.

```
/* Here, Box defines three constructors to initialize
 the dimensions of a box various ways.
*/
class Box {
      double width;
      double height;
      double depth;
}
```

```
// constructor used when all dimensions specified
 Box(double w, double h, double d) {
        width = w;    height = h;    depth = d;
 }
 // constructor used when no dimensions specified
 Box() {
        width = -1; // use -1 to indicate
        height = -1; // an uninitialized
        depth = -1; // box
 }
 // constructor used when cube is created
 Box(double len) {
       width = height = depth = len;
 }
 // compute and return volume
 double volume() {
       return width * height * depth;
 }}
```

```
class OverloadCons {
   public static void main(String args[]) {
    // create boxes using the various constructors
        Box mybox1 = new Box(10, 20, 15);
        Box mybox2 = new Box();
        Box mycube = new Box(7);
        double vol;

        // get volume of first box
        vol = mybox1.volume();
        System.out.println("Volume of mybox1 is " + vol);

        // get volume of second box
        vol = mybox2.volume();
        System.out.println("Volume of mybox2 is " + vol);

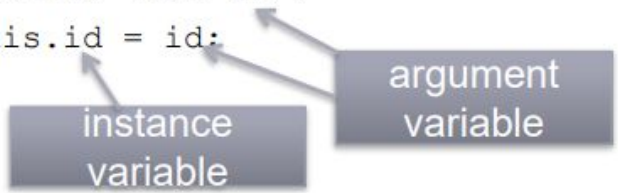
        // get volume of cube
        vol = mycube.volume();
        System.out.println("Volume of mycube is " + vol);
        }
}
```

this

- Each class member function contains an implicit reference of its class type, named this
- this reference is created automatically by the compiler
- It contains the address of the object through which the function is invoked
- Use of this keyword
  - ✔ this can be used to refer instance variables when there is a clash with local variables or method arguments
  - ✔ this can be used to call overloaded constructors from another constructor of the same class

Ex1:

```
void  setId (int id){
    this.id = id;
}
```



Ex2:

```
class Sample{
Sample(){
this("Java"); // calls overloaded constructor
System.out.println("Default constructor ");
}
Sample(String str){
System.out.println("One argument constructor "+ str);
}
}
```

this Reference (Contd.).

- Use this.variableName to explicitly refer to the instance variable.
- Use variable Name to refer to the parameter.
- The this reference is implicitly used to refer to instance variables and methods.
- It CANNOT be used in a static method.

What will be the output of the following code?

```
class Person {
    String name;
    void setName(String n) {
        name = n;
    }
    void printName() {
        System.out.println(name);
    }
}

class Test {
    public static void main(String[] args) {
        Person p = new Person();
        p.printName();
    }
}
```

null

What will be the output of the following code?

```
class Student {
String name;
int age;
Student(String name, int age) {
    this.name = name;
    this.age = age;
}
void display()
{
    System.out.println("Name: " + name + ", Age: " + age);
}
}
public class Main {
    public static void main(String[] args) {
        Student student = new Student("Alice", 20);
        student.display();
    }
}
```

Problem Statement 1

One day, a creative architect received an unusual request from a client. The client needed a software program to calculate the area of rectangle-shaped garden plots. The client had the idea of using rectangular plots with various widths and heights to create unique gardens.

Write a program using a class Rectangle and a constructor to calculate the area of the rectangle.

Formula:
area of the rectangle = width * height
Input Format :
The input consists of two space-separated double values, representing the width and height, respectively.
Output Format :
The output prints the area of the rectangle, rounded off to two decimal places.

Solution

```
class Rectangle {
    private double width;
    private double height;
    private double area;

    public Rectangle(double width, double height) {
        width = width;
        height = height;
        area = width * height;
    }

    public double getArea() {
        return area;
    }
}
```

```
class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        double width = scanner.nextDouble();
        double height = scanner.nextDouble();

        Rectangle rectangle = new Rectangle(width, height);
        double area = rectangle.getArea();
        System.out.printf("%.2f" ,area);
        scanner.close();
    }
}
```

Problem Statement 2

Alen is tasked with creating a Java program that functions as a power calculator. The program should define a PowerCalculator class that takes a base and an exponent as parameters using a constructor. The class should calculate the result of raising the base to the given exponent. In the case where the base is zero and the exponent is negative, the program should indicate that the result is undefined.

Class definition:

Create a 'PowerCalculator' class

Create a main class, that will handle user interactions and invoke the 'PowerCalculator' class.

Input Format :

The input consists of the following on separate lines:

The base value (a double)

The exponent (an integer)

Output Format :

The output should display the following:

The result of raising the base to the exponent in the following format: "**<Base value> raised to the power of <exponent value> is <calculated value>**", where the result is a double value.

If the base is zero and the exponent is negative, the program should print "**Result is undefined. The base is zero, and the exponent is negative**"

Solution

```
class PowerCalculator
{
    double a;
    int b;
    PowerCalculator(double a,int b){
        this.a = a;
        this.b = b;
    }
    public void displayResult(){
        if(a==0 && b<0){
            System.out.println("Result is undefined. Base is zero, and the exponent is negative ");
        }else
            System.out.println(a+" raised to the power of"+b+" is "+Math.pow(a,b));
    }
}
```

Problem Statement 3

John is curious to know how many leap years he has lived through. He wants a program where he can input his birth year and the current year, and the program calculates the total number of leap years between them.

Leap Year Conditions:
A year is considered a leap year if:
It is divisible by 4, and
It is not divisible by 100 unless it is also divisible by 400.

Implement a class AgeCalculatorFunctions with a constructor AgeCalculatorFunctions(int birthYear, int currentYear) to initialize the birth year. The class should contain a method to count the number of leap years from the birth year to the current year.

Input Format :
The first line contains an integer **birthYear**, representing Alex's birth year.
The second line contains an integer **currentYear**, representing the current year.

Output Format :
The output displays an **integer**, representing the total number of leap years Alex has experienced.

Solution

```
class AgeCalculatorFunctions {
   private int birthYear;
   private int currentYear;

   public AgeCalculatorFunctions(int birthYear, int currentYear) {
      this.birthYear = birthYear;
      this.currentYear = currentYear;
   }
   public int calculateLeapYears() {
      int leapYearCount = 0;
      for (int year = birthYear; year <= currentYear; year++) {
         if (year % 4 == 0 && (year % 100 != 0 || year % 400 == 0)) {
            leapYearCount++;
         }
      }
      return leapYearCount;
   }
}
```

Problem Statement 4

Goutham is given the task of developing a program that efficiently shuffles the elements of an array. He wants to create an ArrayRotator class with a constructor that takes an array and the number of positions to rotate to the right. Users input the array size and elements, specify the rotating positions, and print the rearranged sequence.

Assist Goutham in developing the program.

Input Format :

The first line of input consists of an integer **N,** representing the elements of the array.

The second line consists of **N** space-separated integers representing the array elements.

The third line consists of an integer **M,** representing the number of positions to rotate.

Output Format :

The output prints the rearranged array elements.

Testcase 1 Input

7

3 6 9 8 5 2 1

3

Testcase 1 Output

5 2 1 3 6 9 8

Solution

```
class ArrayRotator {
    private int[] rotatedArray;
    public ArrayRotator(int[] originalArray, int positions) {
        int length = originalArray.length;
        rotatedArray = new int[length];
        int effectivePositions = positions % length;
        for (int i = 0; i < length; i++) {
            int newPosition = (i + effectivePositions) % length;
            rotatedArray[newPosition] = originalArray[i];
        }    }
    public void printRotatedArray() {
        for (int i = 0; i < rotatedArray.length; i++) {
            System.out.print(rotatedArray[i]);
            if (i < rotatedArray.length - 1) {
                System.out.print(" ");
            }
        }
    }
}
```



Thankyou